AgenticTM

Documentación Completa v0.3.2

Motor: Pipeline multi-agente de Threat Modeling con LangGraph + Ollama

Febrero 2026

Contents

Prólogo y Stack Tecnológico **1**

1 Introducción y Visión **2**
1.1 ¿Qué es AgenticTM? . . . 2
1.2 Origen e Inspiración . . . 2
1.3 ¿Que Problema Resuelve? . . . 3
1.4 Diferenciadores Clave . . . 4
1.5 Requisitos de Sistema . . . 4

2 Fundamentos Académicos **6**
2.1 Panorama . . . 6
2.2 STRIDE (Microsoft, 1999) . . . 7
2.3 PASTA — Process for Attack Simulation and Threat Analysis (VerSprite, 2012) . . . 7
2.4 Attack Trees (Schneier, 1999) . . . 8
2.5 DREAD (Microsoft, 2002) . . . 9
2.6 MAESTRO — Multi-Agent Environment for Security Threat Risk Observations (CSA, 2024) . . . 10
2.7 OWASP Top 10 para LLM Applications (2025) . . . 10
2.8 OWASP Agentic AI Top 10 (2026) . . . 11
2.9 PLOT4ai (2023) . . . 11
2.10 AI Agent Protocol Security Taxonomy (CIC/UNB, 2026) . . . 11
2.11 Bases de Datos de Amenazas (via RAG) . . . 12
2.12 Convergencia en AgenticTM . . . 13

3 Arquitectura del Pipeline **14**
3.1 Vision General . . . 14
3.2 Las 6 Fases . . . 14
3.3 Estado Compartido: `ThreatModelState` . . . 15
3.4 Modos de Ejecucion de Analistas . . . 17
3.5 Debate Loop y Convergencia . . . 18
3.6 Error Handling: `_safe_node` . . . 19
3.7 Asignacion de Modelos por Nodo . . . 20
3.8 Construccion del Grafo . . . 20
3.9 Topologia Detallada del Grafo . . . 21
3.10 Metricas por Agente . . . 22

4 Agentes en Profundidad **24**
4.1 Inventario de Agentes . . . 24
4.2 Infraestructura Comun: `base.py` (755 lineas) . . . 24
4.3 1. Architecture Parser (Fase I) . . . 25
4.4 2. STRIDE Analyst (Fase II) . . . 26

4.5 3. PASTA Analyst (Fase II) . . . 27
4.6 4. Attack Tree Analyst (Fase II + Fase II.5) . . . 28
4.7 5. MAESTRO Analyst (Fase II, Condicional) . . . 28
4.8 6. AI Threat Analyst (Fase II, Condicional) . . . 29
4.9 7–8. Red Team y Blue Team Debaters (Fase III) . . . 30
4.10 9. Threat Synthesizer (Fase IV) . . . 31
4.11 10. DREAD Validator (Fase IV) . . . 33
4.12 11. Output Localizer (Fase V) . . . 33
4.13 12. Report Generator (Fase V) . . . 34
4.14 Resumen de herramientas RAG por agente . . . 35

5 Sistema RAG **36**
5.1 Vision General . . . 36
5.2 Knowledge Base: Estructura . . . 36
5.3 ChromaDB: Busqueda Vectorial . . . 37
5.4 PageIndex: Navegacion Estructural . . . 38
5.5 Herramientas RAG (`tools.py`, 92 lineas) . . . 39
5.6 Categorias (`categories.py`, 151 lineas) . . . 40
5.7 `RAGStoreManager` (175 lineas) . . . 41
5.8 Indexacion: Flujo Completo . . . 42
5.9 Datos: 14 Tree Indices . . . 42
5.10 Limitaciones Conocidas . . . 43

6 Modelos LLM **44**
6.1 Arquitectura de 4 Tiers . . . 44
6.2 LLMFactory (`agentictm/llm/__init__.py`, 165 lineas) . . . 45
6.3 5 Providers Soportados . . . 46
6.4 Modelos Ollama Requeridos . . . 47
6.5 Recomendaciones de Hardware . . . 48
6.6 Timeouts . . . 48

7 API y Frontend **49**
7.1 Vision General . . . 49
7.2 Rutas API — Referencia Completa . . . 49
7.3 SSE Streaming — Implementacion . . . 51
7.4 Autenticacion . . . 51
7.5 Persistencia — 3 Capas . . . 52
7.6 Manejo de Uploads . . . 52
7.7 Frontend SPA . . . 52
7.8 Dependencias CDN . . . 53
7.9 Lifespan y middleware . . . 53
7.10 Metricas de Observabilidad . . . 53
7.11 Ejemplo: Flujo Completo de Analisis . . . 54

8 Configuracion **55**
8.1 Archivo `config.json` . . . 55
8.2 Referencia por seccion . . . 56
8.3 Variables de entorno . . . 58
8.4 Modelos Pydantic . . . 59
8.5 `validate_startup()` — validacion al inicio . . . 59
8.6 Configuracion para Docker . . . 60
8.7 Configuraciones recomendadas . . . 60

9 Guia de Uso **62**
9.1 Requisitos Previos . . . 62
9.2 Instalacion . . . 62
9.3 Uso por CLI . . . 63
9.4 Uso por interfaz web . . . 64
9.5 Docker . . . 65
9.6 Flujo completo: de cero a threat model . . . 66
9.7 Buenas practicas . . . 66

10 Flujo de Datos **67**
10.1 `ThreatModelState` — El Bus de Datos . . . 67
10.2 Mapa Lectura/Escritura por Agente . . . 68
10.3 Flujo por Fase . . . 69
10.4 Diagrama completo: estado a lo largo del pipeline . . . 70
10.5 Volumen de Datos por Fase . . . 70
10.6 Manejo de errores en el flujo . . . 71

11 Mejoras y Roadmap **72**
11.1 Estado del Proyecto . . . 72
11.2 Mejoras Implementadas . . . 72
11.3 Mejoras Pendientes . . . 73
11.4 Hallazgos de Evaluaciones . . . 74
11.5 Vision Oraculus — Enterprise Future . . . 74
11.6 Proximos pasos recomendados . . . 75

12 Troubleshooting **76**
12.1 Errores Comunes . . . 76
12.2 Estructura de Archivos del Proyecto . . . 76

13 Evaluación Profunda de AgenticTM y Plan de Acción **78**
13.1 Veredicto Resumido . . . 78
13.2 Deudas Técnicas Prioritarias (P0 y P1) . . . 78

14 Análisis Objetivo y Roadmap de Evolución **79**
14.1 Análisis de Componentes . . . 79
14.2 Roadmap Estratégico . . . 79

15 Evaluación Independiente de AgenticTM **80**
15.1 Análisis de Seguridad y Hardening . . . 80
15.2 Observabilidad y UX . . . 80

16 Lista Priorizada de Mejoras **81**
16.1 Leyenda de Prioridad . . . 81
16.2 Checklist de Tareas . . . 81

Prólogo y Stack Tecnológico

El presente documento consolida la documentación oficial del proyecto **AgenticTM** (versión 0.3.2). Incluye desde la introducción y visión, pasando por la arquitectura del pipeline, detalles de agentes, hasta la guía de instalación y uso.

Nota: Los capítulos correspondientes al Flujo de Datos, Mejoras/Roadmap y Troubleshooting (índices 10 al 12) están pendientes y se añadirán en la próxima revisión documental.

Stack Tecnológico

- **Frontend:** SPA HTML/JS, Mermaid v11, marked.js
- **API:** FastAPI, SSE, aiosqlite, uvicorn
- **Pipeline:** LangGraph, 13 nodos, 6 fases
- **LLMs:** Ollama (local), 4 tiers de modelo
- **RAG:** ChromaDB, PageIndex trees, nomic-embed
- **Lenguaje:** Python 3.13, Pydantic v2, Typer CLI
- **Deploy:** Docker, docker-compose, CI/CD GitHub

Chapter 1

Introducción y Visión

01 – Introducción y Visión

> *"Lo mejor de todos los mundos"* — Un analista experto por cada metodología de threat modeling, debatiendo entre sí para producir un resultado más completo que cualquier enfoque individual.

1.1 ¿Qué es AgenticTM?

AgenticTM (Agentic Threat Modeler) es un sistema multi-agente de threat modeling que orquesta **12 agentes especializados** de inteligencia artificial para analizar la arquitectura de un sistema de software y producir un modelo de amenazas profesional.

A diferencia de las herramientas tradicionales que dependen de un solo modelo o un checklist estático, AgenticTM despliega un **equipo virtual de expertos en seguridad**: cada agente domina una metodología específica (STRIDE, PASTA, Attack Trees, MAESTRO, OWASP AI), y luego un **debate adversarial** (Red Team vs Blue Team) valida, escala y refina las amenazas encontradas.

El resultado es un **Threat Model completo** con:

- CSV de amenazas con scoring DREAD (compatible con plantillas organizacionales).
- Reporte Markdown con resumen ejecutivo, DFD, tablas detalladas y debate.
- Reporte LaTeX profesional listo para compilar.
- Diagramas DFD en Mermaid.
- Arboles de ataque visuales.

1.2 Origen e Inspiración

AgenticTM nace de la convergencia de tres lineas de trabajo:

1. Necesidad Operativa

El equipo de AppSec realizaba threat models manuales que consumian **3–5 dias por sistema**. Con docenas de proyectos en pipeline, la carga operativa era insostenible. Se necesitaba una herramienta que transformara a AppSec de "operador" a "habilitador" manteniendo la profundidad de analisis pero reduciendo el tiempo a **minutos**.

2. Investigacion Academica

Multiples papers recientes demuestran que los LLMs pueden identificar amenazas de seguridad con cobertura comparable a analistas humanos, especialmente cuando se combinan con RAG (Retrieval-Augmented Generation) y frameworks estructurados como STRIDE y PASTA.

3. Arquitectura TradingAgents

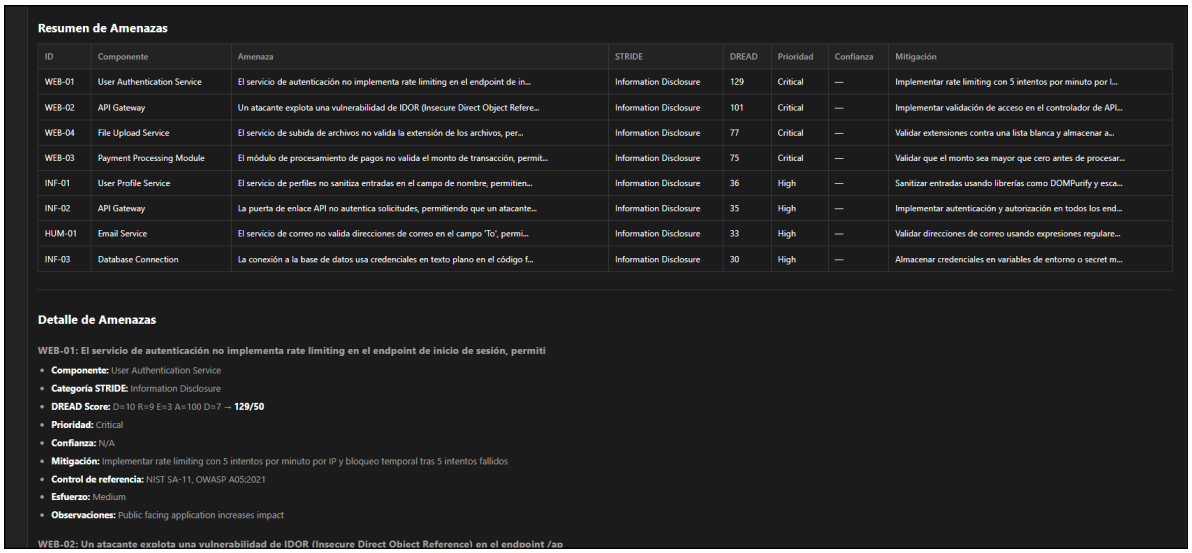
El diseño del pipeline se inspira en TradingAgents v0.4.0 (`https://github.com/TradingAgents`):

- Fan-out paralelo de analistas especializados.
- Debate adversarial (Bull vs Bear a Red vs Blue).
- Patron Quick/Deep Thinker (modelos rapidos para triage, modelos profundos para sintesis).
- Orquestacion LangGraph con estado compartido tipado.

AgenticTM adapta esta arquitectura: analistas financieros a analistas de seguridad, Bull/Bear a Red/Blue Team, Research Manager a Threat Synthesizer.

1.3 ¿Que Problema Resuelve?

```
graph LR
    A["Documento de arquitectura"] --> B["AgenticTM 12 agentes + RAG"]
    B --> C["CSV de amenazas STRIDE + DREAD"]
    B --> D["Reporte completo Markdown + LaTeX"]
    B --> E["Diagramas DFD Mermaid"]
    B --> F["Arboles de ataque visuales"]
```



Resumen de Amenazas

ID	Componente	Amenaza	STRIDE	DREAD	Prioridad	Confianza	Mitigación
WEB-01	User Authentication Service	El servicio de autenticación no implementa rate limiting en el endpoint de in...	Information Disclosure	129	Critical	—	Implementar rate limiting con 5 intentos por minuto por I...
WEB-02	API Gateway	Un atacante explota una vulnerabilidad de IDOR (Insecure Direct Object Refere...	Information Disclosure	101	Critical	—	Implementar validación de acceso en el controlador de API...
WEB-04	File Upload Service	El servicio de subida de archivos no valida la extensión de los archivos, per...	Information Disclosure	77	Critical	—	Validar extensiones contra una lista blanca y almacenar a...
WEB-03	Payment Processing Module	El módulo de procesamiento de pagos no valida el monto de transacción, permit...	Information Disclosure	75	Critical	—	Validar que el monto sea mayor que cero antes de procesar...
INF-01	User Profile Service	El servicio de perfiles no sanitiza entradas en el campo de nombre, permitien...	Information Disclosure	36	High	—	Sanitizar entradas usando librerías como DOMPurify y esca...
INF-02	API Gateway	La puerta de enlace API no autentica solicitudes, permitiendo que un atacante...	Information Disclosure	35	High	—	Implementar autenticación y autorización en todos los end...
HUM-01	Email Service	El servicio de correo no valida direcciones de correo en el campo 'To', permi...	Information Disclosure	33	High	—	Validar direcciones de correo usando expresiones regulare...
INF-03	Database Connection	La conexión a la base de datos usa credenciales en texto plano en el código f...	Information Disclosure	30	High	—	Almacenar credenciales en variables de entorno o secret m...

Detalle de Amenazas

WEB-01: El servicio de autenticación no implementa rate limiting en el endpoint de inicio de sesión, permiti

- **Componente:** User Authentication Service
- **Categoría STRIDE:** Information Disclosure
- **DREAD Score:** D=10 R=9 E=3 A=100 D=7 → **129/50**
- **Prioridad:** Critical
- **Confianza:** N/A
- **Mitigación:** Implementar rate limiting con 5 intentos por minuto por IP y bloqueo temporal tras 5 intentos fallidos
- **Control de referencia:** NIST SA-11, OWASP A05:2021
- **Esfuerzo:** Medium
- **Observaciones:** Public facing application increases impact

WEB-02: Un atacante explota una vulnerabilidad de IDOR (Insecure Direct Object Reference) en el endpoint /ap

Figure 1.1: Resumen visual del sistema AgenticTM.

Problema	Como lo resuelve AgenticTM
TM manual toma 3–5 dias	Pipeline completo en 15–45 minutos.
Un solo analista = una sola perspectiva	5 metodologias diferentes en paralelo.
Sin cobertura de amenazas AI/LLM	Agentes en MAESTRO, OWASP AI, PLOT4ai, Agent Protocol Security.
Inconsistencia entre TMs	RAG con TMs previos como few-shot examples.
Sin debate/validacion cruzada	Red Team vs Blue Team adversarial.
Output no aprovechable	CSV compatible con templates organizacionales, reporte en Markdown/LaTeX.
Requiere GPU cloud costosa	Corre 100% local con Ollama (16–32 GB RAM).

1.4 Diferenciadores Clave

Multi-Metodologia Simultanea

No usa solo STRIDE ni solo PASTA. Ejecuta **5 metodologias en paralelo** y despues sintetiza lo mejor de cada una. Esto produce una cobertura de amenazas significativamente mayor que cualquier enfoque individual.

Debate Adversarial

Luego del analisis, un Red Team (atacante) debate contra un Blue Team (defensor) durante N rondas. El Red Team escala amenazas y propone ataques compuestos; el Blue Team evalua controles existentes y propone mitigaciones. El debate converge cuando no se encuentran vectores nuevos (`[CONVERGENCIA]`).

RAG con Knowledge Base Dual

El sistema combina **busqueda vectorial** (ChromaDB) con **navegacion estructural** (PageIndex trees) para consultar libros de seguridad, papers, bases de riesgos/mitigaciones, threat models previos y amenazas de AI, todo indexado localmente.

100% Local

Todo el procesamiento ocurre en la maquina del usuario via **Ollama**. Ningun dato sale del entorno local, critico para organizaciones con requisitos de soberania de datos.

Condicional e Inteligente

Los agentes MAESTRO y AI Threat se activan solo si el sistema analizado contiene componentes de AI/ML (deteccion por keywords). Esto evita falsos positivos innecesarios en sistemas tradicionales.

1.5 Requisitos de Sistema

Hardware minimo

Recurso	Minimo	Recomendado
RAM	16 GB	32 GB
VRAM (GPU)	8 GB	16+ GB
Disco	20 GB libres	50+ GB (para modelos Ollama)
CPU	8 cores	16+ cores

Software

Componente	Version
Python	3.11+ (recomendado 3.13)
Ollama	Latest
Git	Latest
Docker (opcional)	24+ con Compose v2

Modelos Ollama requeridos

Descarga de modelos Ollama

```
ollama pull qwen3:8b # Quick Thinker (analistas rapidos)
ollama pull qwen3:30b-a3b # Deep Thinker (sintesis, MoE)
ollama pull deepseek-r1:14b # STRIDE + Debate (Chain-of-Thought)
ollama pull qwen3-vl:8b # Vision (diagramas/imagenes)
ollama pull nomic-embed-text # Embeddings para RAG
```

Nota sobre VRAM: con `max_parallel_analysts: 2` (default), el sistema carga como maximo 2 modelos de analisis simultaneamente. En una GPU de 16 GB, esto funciona bien con `qwen3:8b` (aprox. 5 GB cada uno). El `qwen3:30b-a3b` usa MoE con solo 3.3B de parametros activos, pero requiere aprox. 18 GB de disco y 8 GB de VRAM efectiva.

Chapter 2

Fundamentos Académicos

02 – Fundamentos Academicos

Los frameworks y papers de investigacion que alimentan la inteligencia de AgenticTM.

2.1 Panorama

AgenticTM no implementa una sola metodologia. Integra **6 frameworks de threat modeling**, **4 taxonomias de amenazas AI** y **3 bases de datos de ataque** en un pipeline unificado. Cada agente especializado domina una porcion de este corpus, y el Threat Synthesizer consolida todo en un modelo de amenazas cohesivo.

```
graph TB
    subgraph "Frameworks Clasicos"
        STRIDE["STRIDE (Microsoft, 1999)"]
        PASTA["PASTA (VerSprite, 2012)"]
        ATREE["Attack Trees (Schneier, 1999)"]
        DREAD["DREAD (Microsoft, 2002)"]
    end
    subgraph "Frameworks AI/Agenticos"
        MAESTRO["MAESTRO 7-Layer (CSA, 2024)"]
        OWASP_LLM["OWASP Top 10 LLM (2025)"]
        OWASP_AGE["OWASP Agentic AI Top 10 (2026)"]
        PLOT4AI["PLOT4ai (2023)"]
        PROTO["Agent Protocol Security (CIC/UNB, 2026)"]
    end
    subgraph "Bases de Datos"
        CAPEC["CAPEC (MITRE)"]
        CWE["CWE (MITRE)"]
        ATT["MITRE ATT&CK"]
        NIST["NIST SP 800-53"]
    end
    STRIDE --> SYN["Threat Synthesizer"]
    PASTA --> SYN
    ATREE --> SYN
    MAESTRO --> SYN
    OWASP_LLM --> SYN
    OWASP_AGE --> SYN
    PLOT4AI --> SYN
    PROTO --> SYN
    DREAD --> SYN
```

```
CAPEC -.-> SYN
CWE -.-> SYN
ATT -.-> SYN
NIST -.-> SYN
```

2.2 STRIDE (Microsoft, 1999)

Concepto

STRIDE es un modelo de clasificacion de amenazas desarrollado por Loren Kohnfelder y Praerit Garg en Microsoft. El acronimo representa seis categorias:

Categoria	Propiedad violada	Descripcion
S (Spoofing)	Autenticacion	Suplantar la identidad de otro componente o usuario.
T (Tampering)	Integridad	Modificar datos en transito o reposo sin autorizacion.
R (Repudiation)	No-repudio	Negar haber realizado una accion sin poder ser refutado.
I (Information Disclosure)	Confidencialidad	Exponer informacion a actores no autorizados.
D (Denial of Service)	Disponibilidad	Degradar o interrumpir el servicio.
E (Elevation of Privilege)	Autorizacion	Obtener permisos superiores a los asignados.

Implementacion en AgenticTM

El **STRIDE Analyst** (`stride_analyst.py`) aplica STRIDE-per-element: examina cada componente, flujo de datos y boundary del sistema, y evalua cual de las 6 categorias aplica. Usa el tier `stride_json` (DeepSeek-R1:14b) para producir un audit trail visible.

```
{
  "methodology": "STRIDE",
  "threats": [
    {
      "component": "API Gateway",
      "stride_category": "S",
      "description": "Un atacante podria forjar tokens JWT...",
      "impact": "High",
      "references": "CAPEC-94, CWE-287, ATT&CK T1078",
      "confidence_score": 0.85
    }
  ]
}
```

Referencias academicas

- Shostack, A. (2014). *Threat Modeling: Designing for Security*. Wiley.
- Kohnfelder, L. y Garg, P. (1999). *The Threats to our Products*. Memo interno.

2.3 PASTA — Process for Attack Simulation and Threat Analysis (VerSprite, 2012)

Concepto

PASTA es una metodologia de threat modeling centrada en el **riesgo de negocio**, con 7 etapas:

Etapa	Nombre	Proposito
1	Definicion de objetivos	Establecer contexto de negocio y compliance.
2	Alcance tecnico	Inventariar componentes y tecnologias.
3	Descomposicion	Mapear DFDs, trust boundaries, entry points.
4	Analisis de amenazas	Identificar escenarios basados en intelligence.
5	Analisis de vulnerabilidades	Mapear CVEs, CWEs y debilidades conocidas.
6	Modelado de ataques	Simular ataques y construir arboles de ataque.
7	Analisis de riesgo e impacto	Cuantificar riesgo y priorizar remediacion.

Implementacion en AgenticTM

El **PASTA Analyst** (`pasta_analyst.py`) condensa las 7 etapas en una sola invocacion LLM que produce narrativas de ataque de 3–5 oraciones.

```
{
  "methodology": "PASTA",
  "business_context": "Sistema financiero multi-tenant con datos PCI-DSS...",
  "threats": [
    {
      "attack_scenario": "Un atacante con credenciales de merchant...",
      "target_asset": "Ledger de transacciones",
      "attack_path": ["1. Obtener acceso...", "2. Explotar...", "3. Exfiltrar..."],
      "risk_level": "High",
      "vulnerabilities_exploited": "CWE-863, CVE-2024-XXXX"
    }
  ]
}
```

Diferencia con STRIDE

STRIDE clasifica por tipo de amenaza; PASTA clasifica por escenario de ataque. AgenticTM usa ambas perspectivas para maximizar cobertura.

Referencias

UcedaVelez, T. y Morana, M. (2015). *Risk Centric Threat Modeling*. Wiley.

2.4 Attack Trees (Schneier, 1999)

Concepto

Un Attack Tree es una estructura jerarquica donde la raiz es el objetivo del atacante, los nodos intermedios representan sub-objetivos con operadores AND/OR y las hojas son acciones concretas con dificultad o costo asociado. El *cheapest path* es el conjunto de acciones con menor dificultad total que logra el objetivo.

Implementacion en AgenticTM

El **Attack Tree Analyst** tiene dos pasadas: una inicial en Fase II y otra enriquecida en Fase II.5 con acceso a STRIDE, PASTA, MAESTRO, AI Threats y debate Red/Blue.

```
graph TD
    ROOT["Exfiltrar PII de usuarios"]
    ROOT --> OR1["OR"]
    OR1 --> A["SQL Injection en API publica (baja)"]
    OR1 --> B["Credenciales admin comprometidas (media)"]
    OR1 --> AND1["AND"]
    AND1 --> C["Phishing a empleado (baja)"]
    AND1 --> D["Escalar privilegios en red interna (alta)"]
```

Referencias

- Schneier, B. (1999). *Attack Trees*. Dr. Dobb's Journal.
- Mauw, S. y Oostdijk, M. (2005). *Foundations of Attack Trees*. ISSA.

2.5 DREAD (Microsoft, 2002)

Concepto

DREAD es un sistema de scoring cuantitativo. Cada dimension se puntua de 0 a 10:

Dimension	Pregunta
Damage	Que tan grave es el impacto si se explota.
Reproducibility	Que tan facil es reproducir el ataque.
Exploitability	Que tan facil es ejecutar el ataque.
Affected Users	Cuantos usuarios se ven impactados.
Discoverability	Que tan facil es descubrir la vulnerabilidad.

Rango	Prioridad
40–50	Critical
25–39	High
11–24	Medium
1–10	Low

Implementacion en AgenticTM

El scoring se asigna en dos etapas: **Threat Synthesizer** (scores iniciales) y **DREAD Validator** (validacion y ajuste). Si el validador devuelve menos del 80% de amenazas, se hace un merge para preservar faltantes con scores originales.

Formula

$$\text{DREAD Total} = D + R + E + A + D$$

$$\text{Priority} = \begin{cases} \text{Critical} & \text{si DREAD} \geq 40 \\ \text{High} & \text{si } 25 \leq \text{DREAD} < 40 \\ \text{Medium} & \text{si } 11 \leq \text{DREAD} < 25 \\ \text{Low} & \text{si DREAD} < 11 \end{cases}$$

Notas

DREAD fue descontinuado por Microsoft en 2008 por considerarlo subjetivo. Sin embargo, sigue siendo ampliamente usado en la industria y el doble scoring de AgenticTM mitiga parte de la subjetividad.

2.6 MAESTRO — Multi-Agent Environment for Security Threat Risk Observations (CSA, 2024)

Concepto

MAESTRO es un framework de la Cloud Security Alliance diseñado para amenazas en sistemas AI/ML agenticos. Define 7 capas:

Capa	Nombre	Foco
L1	Foundation Models	Poisoning, jailbreak, hallucination.
L2	Data Operations	Ingestion, preprocessing, storage, quality.
L3	Agent Frameworks	Prompt injection, tool misuse.
L4	Deployment & Infrastructure	Contenedores, orquestacion, model serving.
L5	Tool & API Integration	APIs, MCP servers, credential management.
L6	Agent-to-Agent Communication	Protocolos, trust, message tampering.
L7	Ecosystem & Governance	Compliance, audit trails, versioning.

Implementacion en AgenticTM

El **MAESTRO Analyst** (`maestro_analyst.py`) se activa condicionalmente solo cuando detecta componentes AI/ML. Combina MAESTRO con OWASP Agentic AI Top 10.

```
AI_KEYWORDS = ["ai", "ml", "llm", "model", "agent", "agentic", "rag",
               "embedding", "neural", "prompt", "langchain", ...]

def _has_ai_components(state) -> bool:
    return any(kw in combined_text.lower() for kw in AI_KEYWORDS)
```

Referencias

Cloud Security Alliance (2024). *MAESTRO: A Framework for Analyzing Multi-Agent AI Systems*.

2.7 OWASP Top 10 para LLM Applications (2025)

Catalogo de vulnerabilidades

ID	Vulnerabilidad	Descripcion
LLM01	Prompt Injection	Inyeccion de instrucciones maliciosas.
LLM02	Sensitive Information Disclosure	Filtracion de datos de contexto o entrenamiento.
LLM03	Supply Chain Vulnerabilities	Dependencias comprometidas en frameworks AI.
LLM04	Data and Model Poisoning	Envenenamiento de datos de entrenamiento.
LLM05	Improper Output Handling	Outputs no sanitizados que causan injection.
LLM06	Excessive Agency	Modelo con permisos excesivos sobre herramientas.
LLM07	System Prompt Leakage	Extraccion del system prompt.
LLM08	Vector and Embedding Weaknesses	Ataques a embeddings y vector stores.
LLM09	Misinformation	Hallucinations con impacto de negocio.
LLM10	Unbounded Consumption	DoS por consumo descontrolado.

Implementacion

El **AI Threat Analyst** referencia LLM01–LLM10 en su prompt para clasificar amenazas LLM.

Referencias

OWASP Foundation (2025). *OWASP Top 10 for LLM Applications v2025*.

2.8 OWASP Agentic AI Top 10 (2026)

Catalogo

ID	Vulnerabilidad	Descripcion
ASI01	Prompt Injection (Agentic)	Inyeccion en contexto multi-agente.
ASI02	Sensitive Info Disclosure	Filtracion cross-agent.
ASI03	Supply Chain	Dependencias de tools/plugins comprometidas.
ASI04	Excessive Agency	Agente con herramientas sin guardrails.
ASI05	Insecure Output Handling	Outputs usados sin sanitizacion.
ASI06	Insufficient Logging	Falta de audit trail.
ASI07	Improper Context Management	Contexto contaminado entre sesiones.
ASI08	Memory Poisoning	Envenenamiento de memoria persistente.
ASI09	Privilege Mismanagement	Escalacion de privilegios cross-agent.
ASI10	Uncontrolled Code Generation	Ejecucion sin sandbox.

Diferencia con OWASP LLM Top 10

OWASP LLM se enfoca en un modelo individual; OWASP Agentic se enfoca en sistemas multi-agente con herramientas, memoria y comunicacion agente-a-agente.

2.9 PLOT4ai (2023)

Concepto

PLOT4ai es una biblioteca de tarjetas con amenazas AI/ML organizadas en 8 categorias:

Categoria	Foco
Data Governance	Gestion de datos de entrenamiento.
Transparency	Explicabilidad y auditabilidad.
Privacy	Proteccion de datos personales.
Cybersecurity	Ataques especificos contra modelos.
Safety	Seguridad fisica y operacional.
Bias/Fairness	Sesgos algoritemicos.
Ethics	Consideraciones eticas de AI.
Accountability/HITL	Supervision humana.

Implementacion en AgenticTM

El deck PLOT4ai (`rag/deck.json`) esta indexado en el vector store `ai_threats`. Los agentes AI Threat y MAESTRO lo consultan via `rag_query_ai_threats()`.

2.10 AI Agent Protocol Security Taxonomy (CIC/UNB, 2026)

Concepto

Esta taxonomia cataloga 32 amenazas especificas de protocolos de comunicacion agente-a-agente.

Clase	Amenazas	Ejemplos
Authentication y Access Control	11	Agent Identity Spoofing, Token Replay, Capability Leak.
Supply Chain e Ecosystem Integrity	9	Malicious Tool Injection, Dependency Confusion.
Operational Integrity y Reliability	12	Context Window Overflow, Response Manipulation.

Protocolos analizados: MCP (Model Context Protocol), A2A (Agent-to-Agent), Agora y ANP.

Implementacion en AgenticTM

El **AI Threat Analyst** detecta dinamicamente protocolos presentes y agrega instrucciones al prompt:

```
PROTOCOL_KEYWORDS = {
  "MCP": ["mcp", "model context protocol", "tool_server", "mcp_server"],
  "A2A": ["a2a", "agent-to-agent", "agent2agent", "google a2a"],
  "Agora": ["agora", "agora protocol"],
  "ANP": ["anp", "agent network protocol"]
}
```

Metricas cuantitativas

- WEI: `attack_complexity * business_impact * layer_weight` (escala 0–10).
- RPS: capas downstream afectadas (1–7, mayor a 3.0 es critico).
- VR: probabilidad de misbinding bajo tool ambiguity (0.0–1.0, mayor a 0.5 es critico).

2.11 Bases de Datos de Amenazas (via RAG)

CAPEC

Patrones de ataque abstractos referenciados por los agentes:

```
CAPEC-94:  Man in the Middle
CAPEC-66:  SQL Injection
CAPEC-112: Brute Force
CAPEC-242: Code Injection
```

CWE

Clasifica debilidades de software:

```
CWE-287: Improper Authentication
CWE-79:  Improper Neutralization of Input (XSS)
CWE-89:  SQL Injection
CWE-863: Incorrect Authorization
```

MITRE ATT&CK

Framework de tacticas y tecnicas adversarias:

```
T1078: Valid Accounts
T1190: Exploit Public-Facing Application
T1071: Application Layer Protocol
```

NIST SP 800-53

Controles de seguridad usados como referencia de mitigaciones:

```
AC-2:  Account Management
AU-3:  Content of Audit Records
SC-8:  Transmission Confidentiality
IA-2:  Identification and Authentication
```

2.12 Convergencia en AgenticTM

La clave de AgenticTM no es usar un framework aislado, sino su convergencia:

1. STRIDE identifica amenazas por categoria abstracta.
2. PASTA agrega contexto de negocio y narrativas de ataque.
3. Attack Trees revelan caminos de menor resistencia.
4. MAESTRO + OWASP + PLOT4ai cubren amenazas AI/agenticas.
5. Agent Protocol Security cubre amenazas de protocolos emergentes.
6. El debate Red/Blue filtra falsos positivos y escala amenazas reales.
7. El Synthesizer consolida 15–40 amenazas con scoring DREAD.
8. NIST/CAPEC/CWE/ATT&CK aportan evidencia y referencias via RAG.

Esta convergencia produce un threat model mas completo, diverso y defendible que cualquier metodologia individual.

Chapter 3

Arquitectura del Pipeline

03 – Arquitectura del Pipeline

13 nodos, 6 fases, 3 modos de ejecucion: la columna vertebral de AgenticTM.

3.1 Vision General

El pipeline de AgenticTM es un **grafo dirigido aciclico** con un ciclo condicional (debate loop), implementado sobre LangGraph (`https://github.com/langchain-ai/langgraph`). LangGraph provee:

- Estado tipado (`ThreatModelState`) compartido entre todos los nodos.
- Fan-out paralelo con barrera de sincronizacion.
- Aristas condicionales para el loop de debate.
- Acumulacion automatica de listas via `Annotated[list,operator.add]`.

3.2 Las 6 Fases

```
graph TD
    subgraph "Fase I - Ingesta"
        AP["Architecture Parser (quick_json + VLM)"]
    end
    subgraph "Fase II - Analisis por Metodologia"
        S["STRIDE Analyst (stride_json)"]
        P["PASTA Analyst (quick_json)"]
        AT["Attack Tree Analyst (quick_json)"]
        M["MAESTRO Analyst (quick_json)"]
        AI["AI Threat Analyst (quick_json)"]
    end
    subgraph "Fase III - Debate Adversarial"
        RED["Red Team (stride CoT)"]
        BLUE["Blue Team (stride CoT)"]
    end
    subgraph "Fase II.5 - Enriquecimiento"
        ATE["Attack Tree Enriched (deep_json)"]
    end
    subgraph "Fase IV - Sintesis y Validacion"
        TS["Threat Synthesizer (deep_json)"]
        DV["DREAD Validator (quick_json)"]
```

```
    end
    subgraph "Fase V - Output"
        OL["Output Localizer (quick_json)"]
        RG["Report Generator (sin LLM)"]
    end
    AP --> S & P & AT & M & AI
    S & P & AT & M & AI --> RED
    RED --> BLUE
    BLUE -->|"mas rondas"| RED
    BLUE -->|"convergencia"| ATE
    ATE --> TS
    TS --> DV
    DV --> OL
    OL --> RG
    RG --> FIN["END"]
```

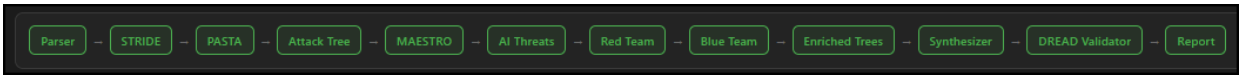


Figure 3.1: Orden de ejecucion y sincronizacion del pipeline multi-agente.

Descripcion de cada fase

Fase	Proposito
I – Ingesta	Convierte el input (texto, Mermaid, imagenes, Draw.io) en un modelo de sistema.
II – Analisis	Cada analista aplica su metodologia de forma independiente.
III – Debate	Red Team y Blue Team validan y refinan amenazas en N rondas.
II.5 – Enriquecimiento	Segunda pasada de attack trees con contexto completo.
IV – Sintesis	Consolida todas las amenazas, asigna DREAD y valida scores.
V – Output	Localiza el contenido y genera CSV, Markdown y LaTeX.

3.3 Estado Compartido: `ThreatModelState`

El estado es un `TypedDict` que fluye por todo el grafo. Cada nodo lee lo que necesita y escribe su seccion.

Diagrama de estado

```
classDiagram
    class ThreatModelState {
        +str system_name
        +str analysis_date
        +str raw_input
        +str input_type
        +str system_description
        +list~Component~ components
        +list~DataFlow~ data_flows
        +list~TrustBoundary~ trust_boundaries
        +list~Component~ external_entities
        +list~Component~ data_stores
```

```
    +str scope_notes
    +str mermaid_dfd
    +list~dict~ methodology_reports
    +dict rag_context
    +str previous_tm_context
    +list~str~ threat_categories
    +list~DebateEntry~ debate_history
    +int debate_round
    +int max_debate_rounds
    +list~Threat~ threats_final
    +str executive_summary
    +str csv_output
    +str report_output
    +int iteration_count
    +dict validation_result
    +list~dict~ _errors
    +str feedback_context
}
class Component {
    +str name
    +str type
    +str description
    +str scope
}
class DataFlow {
    +str source
    +str destination
    +str protocol
    +str data_type
    +bool bidirectional
}
class Threat {
    +str id
    +str component
    +str description
    +str methodology
    +str stride_category
    +int damage
    +int reproducibility
    +int exploitability
    +int affected_users
    +int discoverability
    +int dread_total
    +str priority
    +str mitigation
    +list evidence_sources
    +float confidence_score
}
class DebateEntry {
    +int round
    +str side
    +str argument
```

```
        +list threat_assessments
    }
    ThreatModelState --> Component
    ThreatModelState --> DataFlow
    ThreatModelState --> Threat
    ThreatModelState --> DebateEntry
```

Campos con acumulacion automatica

Los campos marcados usan `Annotated[list,operator.add]`:

Campos acumulables

```python
methodology_reports: Annotated[list[dict[str, Any]], operator.add]
debate_history: Annotated[list[DebateEntry], operator.add]
_errors: Annotated[list[dict[str, Any]], operator.add]
```

Esto permite que LangGraph concatene listas en la barrera de sincronizacion sin sobreescritura.

3.4 Modos de Ejecucion de Analistas

La Fase II soporta tres modos, configurables en `config.json`.

Modo Hybrid (default)

Config: modo hybrid

```jsonc
"analyst_execution_mode": "hybrid",
"max_parallel_analysts": 2
```

```
graph LR
    AP["Architecture Parser"] --> FAN["Fan-out"]
    FAN --> S["STRIDE"]
    FAN --> P["PASTA"]
    FAN --> AT["Attack Tree"]
    FAN --> M["MAESTRO"]
    FAN --> AI["AI Threat"]
    S --> BARRIER["Fan-in Barrier"]
    P --> BARRIER
    AT --> BARRIER
    M --> BARRIER
    AI --> BARRIER
    SEM["Semaforo max=2"] -.->|"throttle"| S
    SEM -.->|"throttle"| P
    SEM -.->|"throttle"| AT
    SEM -.->|"throttle"| M
    SEM -.->|"throttle"| AI
```

Los 5 analistas se despachan en paralelo, pero un semaforo limita a maximo 2 LLMs activos.

Modo Parallel

Config: modo parallel

```
"analyst_execution_mode": "parallel",
"max_parallel_analysts": 5
```

Paralelo real sin throttling. Requiere alta VRAM (aprox. 25 GB para 5 modelos de 8B).

Modo Cascade

Config: modo cascade

```
"analyst_execution_mode": "cascade"
```

```
graph LR
    AP["Architecture Parser"] --> S["STRIDE"]
    S --> P["PASTA"]
    P --> AT["Attack Tree"]
    AT --> M["MAESTRO"]
    M --> AI["AI Threat"]
    AI --> RED["Red Team"]
```

Secuencial, con menor VRAM y riesgo de bias-drag.

Implementacion del throttling

Semaforo de ejecucion

```python
_analyst_sem = threading.Semaphore(max_concurrent)

def _throttled(fn, name: str):
    def _wrapper(state):
        _analyst_sem.acquire()
        try:
            return fn(state)
        finally:
            _analyst_sem.release()
    return _wrapper
```

El semaforo se aplica antes de `_safe_node` para asegurar liberacion aun con errores.

3.5 Debate Loop y Convergencia

El debate implementa un ciclo condicional controlado por `should_continue_debate()`.

```
stateDiagram-v2
    [*] --> RedTeam: Fase III inicia
    RedTeam --> BlueTeam: Argumento Red
    BlueTeam --> Decision: Argumento Blue
    Decision --> RedTeam: debate_round < max y no [CONVERGENCIA]
    Decision --> AttackTreeEnriched: debate_round >= max o [CONVERGENCIA]
    AttackTreeEnriched --> [*]: Continua a Fase IV
```

Condiciones de salida

- Cap de rondas: `debate_round > max_debate_rounds`.
- Convergencia: el Red Team emite `[CONVERGENCIA]`.
- Nuevos vectores: `[NUEVOS VECTORES]` para seguir explorando.

```
def should_continue_debate(state) -> Literal["red_team", "attack_tree_enriched"]:
    current_round = state.get("debate_round", 1)
    effective_max = state.get("max_debate_rounds", max_rounds)
    if current_round > effective_max:
        return "attack_tree_enriched"
    last_red = next((e for e in reversed(debate_history) if e.get("side") == "red"),
    ↪  None)
    if last_red and "[CONVERGENCIA]" in last_red.get("argument", ""):
        return "attack_tree_enriched"
    return "red_team"
```

3.6 Error Handling: `_safe_node`

Cada nodo esta envuelto en `_safe_node` para degradacion graciosa.

```
def _safe_node(node_fn, node_name: str):
    CRITICAL_NODES = {"architecture_parser", "threat_synthesizer"}
    def wrapper(state):
        try:
            return node_fn(state)
        except Exception as exc:
            if node_name in CRITICAL_NODES:
                raise
            return {"_errors": [{
                "node": node_name,
                "error": str(exc),
                "traceback": traceback.format_exc(),
            }]}
    return wrapper
```

Tipo	Comportamiento ante error
Critico (`architecture_parser`, `threat_synthesizer`)	Re-raise, el pipeline aborta.
No-critico (resto)	Log y `_errors`, el pipeline continua.

3.7 Asignacion de Modelos por Nodo

Nodo	Tier	Modelo default	Justificacion
Architecture Parser	Quick JSON	qwen3:8b	Parsing rapido de arquitectura.
Architecture Parser (imagenes)	VLM	qwen3-vl:8b	Vision para diagramas.
STRIDE Analyst	Stride JSON	deepseek-r1:14b	CoT audit trail.
PASTA Analyst	Quick JSON	qwen3:8b	Narrativas de ataque rapidas.
Attack Tree (initial)	Quick JSON	qwen3:8b	Arboles iniciales.
MAESTRO Analyst	Quick JSON	qwen3:8b	Analisis MAESTRO.
AI Threat Analyst	Quick JSON	qwen3:8b	Analisis AI/agentico.
Red Team	Stride (free-text)	deepseek-r1:14b	CoT visible en debate.
Blue Team	Stride (free-text)	deepseek-r1:14b	CoT visible en debate.
Attack Tree (enriched)	Deep JSON	qwen3:30b-a3b	2da pasada con contexto completo.
Threat Synthesizer	Deep JSON	qwen3:30b-a3b	Sintesis de 15+ amenazas.
DREAD Validator	Quick JSON	qwen3:8b	Scoring rapido.
Output Localizer	Quick JSON	qwen3:8b	Traduccion rapida.
Report Generator	–	–	Codigo puro, sin LLM.

3.8 Construccion del Grafo

El grafo se construye en `agentictm/graph/builder.py` con dos funciones.

`build_graph(config, llm_factory) -> StateGraph`

1. Importa funciones `run_*` de los agentes.
2. Crea instancias LLM via `LLMFactory`.
3. Crea wrappers de nodos y aplica throttling.
4. Aplica error handling con `_safe_node`.
5. Construye el `StateGraph(ThreatModelState)` y agrega aristas.
6. Configura la arista condicional del debate.

`compile_graph(config, llm_factory)`

Llama a `build_graph` y luego `.compile()` para obtener un ejecutable.

Uso desde `core.py`

```
class AgenticTM:
    def __init__(self, config):
        self.llm_factory = LLMFactory(...)
        from agentictm.graph.builder import compile_graph
        self._app = compile_graph(self.config, self.llm_factory)
    def analyze(self, system_input, system_name, ...):
        initial_state = {
```

```
            "system_name": system_name,
            "raw_input": system_input,
            "debate_round": 1,
            "methodology_reports": [],
            "debate_history": [],
        }
        result = self._app.invoke(initial_state)
        return result
```

3.9 Topologia Detallada del Grafo

Modo Parallel/Hybrid

```
Entry Point
    |
    v
architecture_parser
    |
    +---- stride_analyst ----+
    +---- pasta_analyst -----|
    +---- attack_tree_analyst|  (fan-out, throttled)
    +---- maestro_analyst ---|
    +---- ai_threat_analyst -+
                             |
                             v
                      red_team <----+
                          |         |
                          v         |
                      blue_team ----+ (loop condicional)
                          |
                          v
                attack_tree_enriched
                          |
                          v
                threat_synthesizer
                          |
                          v
                  dread_validator
                          |
                          v
                output_localizer
                          |
                          v
                report_generator
                          |
                          v
                         END
```

Modo Cascade

```
Entry Point
    |
```

```
    v
architecture_parser
    |
    v
stride_analyst
    |
    v
pasta_analyst
    |
    v
attack_tree_analyst
    |
    v
maestro_analyst
    |
    v
ai_threat_analyst
    |
    v
red_team <----+
    |         |
    v         |
blue_team ----+ (loop condicional)
    |
    v
attack_tree_enriched
    |
    v
threat_synthesizer
    |
    v
dread_validator
    |
    v
output_localizer
    |
    v
report_generator
    |
    v
END
```

3.10 Metricas por Agente

Cada invocacion registra metricas via `get_agent_metrics()` / `clear_agent_metrics()` en `base.py`:

```
class AgentMetrics:
    agent_name: str
    execution_time_seconds: float
    llm_calls: int
    tool_calls: int
    json_parse_strategy: str
```

```
threats_produced: int
self_reflection_applied: bool
errors: list[str]
```

Actualmente se recopilan durante la ejecucion, pero no se persisten.

Chapter 4

Agentes en Profundidad

04 – Agentes en Profundidad

Cada nodo del pipeline: rol, prompt, inputs/outputs, herramientas RAG y tier de LLM.

4.1 Inventario de Agentes

#	Agente	Archivo	Lineas	LLM Tier	RAG Tools
1	Architecture Parser	`architecture_parser.py`	1064	quick_json + VLM	Ninguna
2	STRIDE Analyst	`stride_analyst.py`	152	stride_json	ANALYST_TOOLS
3	PASTA Analyst	`pasta_analyst.py`	142	quick_json	ANALYST_TOOLS
4	Attack Tree Analyst	`attack_tree_analyst.py`	294	quick_json / deep_json	ANALYST_TOOLS
5	MAESTRO Analyst	`maestro_analyst.py`	191	quick_json	AI_ANALYST_TOOL
6	AI Threat Analyst	`ai_threat_analyst.py`	498	quick_json	AI_ANALYST_TOOL
7	Red Team Debater	`debate.py`	475	stride	DEBATE_TOOLS
8	Blue Team Debater	`debate.py`	475	stride	DEBATE_TOOLS
9	Threat Synthesizer	`threat_synthesizer.py`	832	deep_json	SYNTHESIS_TOOL
10	DREAD Validator	`dread_validator.py`	447	quick_json	VALIDATOR_TOOLS
11	Output Localizer	`output_localizer.py`	178	quick_json	Ninguna
12	Report Generator	`report_generator.py`	731	–	Ninguna

4.2 Infraestructura Comun: `base.py` (755 lineas)

Todos los agentes comparten la infraestructura de invocacion definida en `agentictm/agents/base.py`.

`invoke_agent()` — loop principal

```
flowchart TD
    START["invoke_agent()"] --> BIND["Bind tools al LLM"]
    BIND -->|"exito"| LOOP["Tool-calling loop (max_tool_rounds)"]
    BIND -->|"falla"| FALLBACK["Fallback: invocar tools manualmente y embeber en
    ↪  prompt"]
    FALLBACK --> INVOKE["LLM invoke directo"]
    LOOP --> CHECK{"Hay tool_calls en respuesta?"}
    CHECK -->|"si"| EXEC["Ejecutar herramientas"]
    EXEC --> LOOP
    CHECK -->|"no"| REFLECT{"Self-reflection habilitada?"}
    REFLECT -->|"si"| SR["Critique -> Revise (+1 LLM call)"]
    REFLECT -->|"no"| METRICS["Registrar metricas"]
```

```
    SR --> METRICS
    INVOKE --> METRICS
    METRICS --> RETURN["Retornar texto"]
```

Funciones clave

Funcion	Proposito
`build_messages(system_prompt, human_prompt)`	Construye la lista de mensajes base.
`_invoke_tools_into_prompt(...)`	Fallback para modelos sin tool-calling.
`_llm_invoke_with_retry(...)`	Retry con backoff exponencial via tenacity.
`_strip_think_tags(text)`	Elimina `<think>...</think>` en modelos de razonamiento.
`_self_reflect(...)`	Ciclo critique -> revise para mejorar calidad.
`extract_json_from_response(text)`	Extraccion multi-estrategia de JSON.
`parse_structured_response(...)`	Validacion Pydantic con fallback chain.
`extract_threats_from_markdown(...)`	Parser de amenazas desde Markdown.

Extraccion JSON multi-estrategia

Cuando un LLM devuelve JSON, `extract_json_from_response` intenta cinco estrategias en orden: code blocks, full text, balanced brackets, common fixes y markdown fallback.

Self-reflection (opcional)

Cuando `self_reflection_enabled = True`, cada agente pasa por un ciclo adicional: output original -> critic prompt -> revision prompt -> output mejorado. Agrega latencia, mejora calidad.

4.3 1. Architecture Parser (Fase I)

Archivo: `architecture_parser.py` **LLM:** quick_json + VLM

Rol

Punto de entrada del pipeline. Convierte el input del usuario en un modelo de sistema estructurado (componentes, flujos, trust boundaries, data stores, entidades externas, DFD Mermaid).

Input -> Output

Lee del estado	Escribe al estado
`raw_input`	`input_type`, `system_description`, `components`, `data_flows`, `trust_boundaries`, `external_entities`, `data_stores`, `scope_notes`, `mermaid_dfd`

Deteccion de tipo de input

```
def _detect_input_type(raw_input):
    # Retorna: "text" | "mermaid" | "image" | "drawio" | "mixed"
```

Tipo	Deteccion	Procesamiento
text	Sin marcadores especiales	Text LLM directo.
mermaid	Contiene graph/flowchart/sequenceDiagram	Mermaid parser + Text LLM.
image	Paths a .png/.jpg/.svg/.webp	VLM + consolidacion Text LLM.
drawio	Contiene XML Draw.io	Extraccion de texto + Text LLM.
mixed	Texto + imagenes mezclados	VLM para imagenes + Text LLM.

System prompt (extracto)

El prompt instruye al LLM a producir JSON con 8 secciones: `system_description`, `components`, `data_flows`, `trust_boundaries`, `external_entities`, `data_stores`, `api_endpoints`, `deployment_info`.

Flujo para imagenes (VLM)

```
flowchart TD
    IMG["Imagen detectada"] --> VLM["VLM invocacion (retry progresivo)"]
    VLM -->|"intento 1"| FULL["Prompt completo (8 secciones)"]
    FULL -->|"falla"| SIMPLE["Prompt simplificado (solo componentes)"]
    SIMPLE -->|"falla"| MINIMAL["Prompt minimo (describi lo que ves)"]
    VLM --> NORMALIZE["Normalizar output (camelCase, nested)"]
    NORMALIZE --> CONSOLIDATE["Consolidacion con Text LLM"]
    CONSOLIDATE --> OUTPUT["Modelo estructurado"]
```

Generacion de DFD Mermaid

Al final, `_generate_mermaid_dfd()` genera un diagrama `flowchart LR` con componentes y flujos.

4.4 2. STRIDE Analyst (Fase II)

Archivo: `stride_analyst.py` **LLM:** stride_json (DeepSeek-R1:14b)

Rol

Analiza cada elemento del sistema aplicando STRIDE-per-element y produce un audit trail visible.

Input -> Output

Lee del estado	Escribe al estado
`system_description`, `components`, `data_flows`, `trust_boundaries`, `data_stores`, `scope_notes`	`methodology` (append)

RAG tools: `ANALYST_TOOLS`

`rag_query_books`, `rag_query_research`, `rag_query_risks`, `rag_query_previous_tms`.

System prompt (extracto)

Analisis STRIDE exhaustivo para cada componente, flujo y trust boundary.

Schema de output

```
{
  "methodology": "STRIDE",
  "threats": [{
    "component": "API Gateway",
    "stride_category": "S",
    "description": "Descripcion developer-friendly...",
    "impact": "High|Medium|Low",
    "reasoning": "Razonamiento paso a paso",
    "references": "CAPEC-XX, CWE-XX, ATT&CK Txxxx",
    "evidence_sources": [{"source_type": "rag", "source_name": "...", "excerpt":
    ↪  "..."}],
    "confidence_score": 0.85
  }],
  "summary": "Resumen del analisis"
}
```

4.5 3. PASTA Analyst (Fase II)

Archivo: `pasta_analyst.py` **LLM:** quick_json (Qwen3:8b)

Rol

Analiza desde la perspectiva del atacante con foco en riesgo de negocio. Produce narrativas de ataque.

Input -> Output

Lee del estado	Escril al es-tado
`system_description`, `components`, `data_flows`, `trust_boundaries`, `external_entities`, `scope_notes`	`metho` (ap-pend)

RAG tools: `ANALYST_TOOLS`

System prompt (extracto)

Especialista en PASTA (7 etapas, enfoque risk-centric). Narrativas de ataque developer-friendly.

Schema de output

```
{
  "methodology": "PASTA",
  "business_context": "Contexto de negocio del sistema",
  "threats": [{
    "attack_scenario": "Narrativa de 3-5 oraciones",
    "target_asset": "Ledger de transacciones",
    "attack_path": ["1. Obtener acceso...", "2. Explotar...", "3. Exfiltrar..."],
    "likelihood": "High|Medium|Low",
```

```
    "business_impact": "Descripcion del impacto de negocio",
    "risk_level": "High|Medium|Low",
    "vulnerabilities_exploited": "CWE-XX, CVE-XXXX"
  }]
}
```

4.6 4. Attack Tree Analyst (Fase II + Fase II.5)

Archivo: `attack_tree_analyst.py` **LLM:** quick_json (inicial) / deep_json (enriched)

Rol

Genera arboles de ataque jerarquicos con operadores AND/OR. Dos pasadas: inicial y enriched post-debate.

Input -> Output

Pasada	Lee del estado	Escribe al estado
Inicial	`system_description`, `components`, `data_flows`, `trust_boundaries`, `data_stores`, `scope_notes`	`methodology_reports` (append)
Enriched	Anterior + `methodology_reports`, `debate_history`	`methodology_reports` (append)

System prompt (extracto)

Construccion de arboles Mermaid `graph TD` con objetivos principales y cheapest path.

Schema de output

```
{
  "methodology": "ATTACK_TREE",
  "attack_trees": [{
    "root_goal": "Exfiltrar PII de usuarios",
    "tree_mermaid": "graph TD\n  A[Root] --> B[...]",
    "cheapest_path": "SQL Injection -> Data Exfil",
    "cheapest_path_difficulty": "Low",
    "threats": [{
      "leaf_action": "SQL Injection en API publica",
      "component": "API Gateway",
      "difficulty": "Low",
      "mitre_technique": "T1190"
    }]
  }]
}
```

4.7 5. MAESTRO Analyst (Fase II, Condicional)

Archivo: `maestro_analyst.py` **LLM:** quick_json

Rol

Especialista en amenazas AI usando CSA MAESTRO 7-Layer + OWASP Agentic AI Top 10.

Activacion condicional

```
AI_KEYWORDS = ["ai", "ml", "llm", "model", "gpt", "agent", "agentic", "rag",
               "vector", "embedding", "neural", "prompt", "inference", "training",
               "langchain", "langgraph", "ollama", "openai", "anthropic", ...]

def _has_ai_components(state) -> bool:
    text = (description + component_names + raw_input + reports).lower()
    return any(kw in text for kw in AI_KEYWORDS)
```

Si no hay componentes AI, retorna `"methodology_reports": []`.

Input -> Output

Lee del estado	Escribe al estado
`system_description`, `components`, `data_flows`, `scope_notes`, `raw_input`, `methodology_reports`	`methodolog` (append)

RAG tools: `AI_ANALYST_TOOLS`

Incluye `rag_query_ai_threats` ademas de las tools estandar.

Schema de output

```
{
  "methodology": "MAESTRO",
  "threats": [{
    "component": "RAG Pipeline",
    "maestro_layer": "L2",
    "owasp_id": "ASI07",
    "description": "Context poisoning via malicious document injection...",
    "attack_vector": "Uploaded PDF with embedded prompt injection",
    "evidence_sources": [...],
    "confidence_score": 0.8
  }]
}
```

4.8 6. AI Threat Analyst (Fase II, Condicional)

Archivo: `ai_threat_analyst.py` **LLM:** quick_json

Rol

Analiza amenazas AI usando seis frameworks simultaneamente y solo se activa si hay componentes AI.

Frameworks evaluados

PLOT4ai, OWASP Top 10 LLM 2025, OWASP Agentic AI 2026, CSA MAESTRO, Agent Protocol Security y Context-CIA.

Deteccion dinamica de protocolos

```
PROTOCOL_KEYWORDS = {
  "MCP": ["mcp", "model context protocol", "tool_server"],
  "A2A": ["a2a", "agent-to-agent", "agent2agent"],
  "Agora": ["agora", "agora protocol"],
  "ANP": ["anp", "agent network protocol"]
}
```

Metricas cuantitativas

Metrica	Formula	Escala
WEI	`attack_complexity * business_impact * layer_weight`	0–10
RPS	Capas downstream afectadas	1–7 (>3.0 critico)
VR	Prob. misbinding bajo tool ambiguity	0.0–1.0 (>0.5 critico)

Schema de output (parcial)

```
{
  "methodology": "AI_THREAT",
  "threats": [{
    "framework": "OWASP_AGENTIC_2026",
    "category": "ASI04",
    "maestro_layer": "L5",
    "protocol_threat_class": "Supply Chain & Ecosystem Integrity",
    "wei_score": 7.5,
    "rps_score": 4.2,
    "violation_rate": 0.6,
    "context_cia_impact": {"confidentiality": "High", "integrity": "Medium"},
    "cross_layer_risks": ["L3->L5", "L5->L7"],
    "protocol_risks": [{"protocol": "MCP", "threat": "Tool Injection"}],
    "questions_to_evaluate": ["Que sandbox existe para tool execution?"]
  }]
}
```

4.9 7–8. Red Team y Blue Team Debaters (Fase III)

Archivo: `debate.py` **LLM:** stride (DeepSeek-R1:14b, free-text CoT)

Rol

Debate adversarial de N rondas donde el Red Team ataca y el Blue Team defiende.

Contexto compartido

`_build_full_context()` construye input completo: input original, descripcion, componentes, flujos, reportes de metodologias, arboles iniciales e historial de debate.

Schema de output

Red Team:

```
{
  "threat_assessments": [{
    "threat_id": "TM-003",
    "action": "ESCALATE|CONFIRM|NEW_THREAT",
    "reasoning": "...",
    "proposed_dread_total": 42,
    "attack_chain": "STRIDE-S-001 -> PASTA-002 -> lateral movement"
  }]
}
```

Blue Team:

```
{
  "threat_assessments": [{
    "threat_id": "TM-003",
    "verdict": "CONCEDE|DISPUTE|MODERATE",
    "reasoning": "...",
    "existing_controls": "WAF + rate limiting",
    "mitigation": "Add mutual TLS for internal services",
    "control_reference": "NIST SC-8"
  }]
}
```

Senales de convergencia

`[CONVERGENCIA]` para cerrar, `[NUEVOS VECTORES]` para continuar.

4.10 9. Threat Synthesizer (Fase IV)

Archivo: `threat_synthesizer.py` **LLM:** deep_json (Qwen3:30b-a3b)

Rol

Consolida amenazas de todas las metodologias + debate en un threat model unificado.

Input -> Output

Lee del estado
`system_description`, `components`, `trust_boundaries`, `data_flows`, `methodology_reports`, `debate_history`, p

RAG tools: `SYNTHESIS_TOOLS`

`rag_query_previous_tms`, `rag_query_risks`, `rag_query_ai_threats`.

Estrategia hibrida

```
flowchart TD
    START["Extraer baseline de methodology_reports"] --> LLM["Sintesis LLM (timeout
    ↪  15 min)"]
    LLM -->|"LLM >= 60% baseline"| USE_LLM["Usar resultado LLM"]
    LLM -->|"0 < LLM < 60%"| MERGE["Merge: baseline + LLM unicos"]
    LLM -->|"LLM = 0 o timeout"| USE_BASE["Usar baseline puro"]
    USE_LLM --> IDS["Asignar category-based IDs"]
    MERGE --> IDS
    USE_BASE --> IDS
    IDS --> RETURN["threats_final + executive_summary"]
```

Categorias e IDs

Categoria	Prefijo
Infraestructura y Cumplimiento	INF
Privacidad y Logica de Negocio	PRI
Vulnerabilidades Web y API	WEB
Riesgos de Integracion Agentica	AGE
Amenazas Nativas de IA y LLM	LLM
Factores Humanos y Gobernanza	HUM
Amenazas Generales	GEN

Inferencia STRIDE

```
STRIDE_PATTERNS = {
  "S": ["spoof", "impersonat", "identity", "credential", "authentication bypass",
  ↪  ...],
  "T": ["tamper", "modif", "integrity", "inject", "alter", "corrupt", ...],
  "R": ["repudi", "non-repudiation", "audit", "log", "accountability", ...],
  "I": ["disclos", "leak", "expos", "confidential", "exfiltrat", ...],
  "D": ["denial", "dos", "ddos", "flood", "exhaust", "unavailab", ...],
  "E": ["privilege", "escalat", "authorization", "access control", ...],
}
```

Mitigaciones default por STRIDE

STRIDE	Mitigacion default
S	MFA, credential management (NIST IA-2, OWASP ASVS 2.1).
T	Input validation, integrity checks (NIST SI-7, OWASP ASVS 5.1).
R	Logging y audit trails (NIST AU-3, OWASP ASVS 7.1).
I	Encryption, access controls, data masking (NIST SC-8, OWASP ASVS 8.1).
D	Rate limiting, quotas, redundancy (NIST SC-5, OWASP ASVS 11.1).
E	Least privilege, RBAC, secure defaults (NIST AC-6, OWASP ASVS 4.1).

System prompt (extracto)

Lead Security Architect: consolidar amenazas, minimo 15, incorporar debate, asignar DREAD y mitigaciones mapeadas a NIST 800-53, OWASP ASVS y CIS Controls.

4.11 10. DREAD Validator (Fase IV)

Archivo: `dread_validator.py` **LLM:** quick_json

Rol

Valida y corrige scores DREAD contra la arquitectura real.

Input -> Output

Lee del estado
`threats_final`, `raw_input`, `system_description`, `components`, `data_flows`, `trust_boundaries`, `methodology_`

RAG tools: `VALIDATOR_TOOLS`

`rag_query_previous_tms`, `rag_query_risks`.

Guardrails

- Nunca perder amenazas: si el validador retorna <80% del total, se hace merge.
- Solo actualizar scores y observaciones; el resto se preserva.
- Recalculo de prioridad despues del merge.

System prompt (extracto)

Validar D/R/E/A/D, ajustar over/under-scoring, asegurar consistencia cross-amenaza, sin eliminar.

4.12 11. Output Localizer (Fase V)

Archivo: `output_localizer.py` **LLM:** quick_json

Rol

Traduce contenido a espanol neutro profesional cuando `output_language = "es"`.

Que traduce

Contenido	Se traduce	Razon
`mermaid_dfd` (labels)	Si	Labels de nodos y aristas.
`debate_history`	Si	Argumentos en prosa.
`methodology_reports` (attack tree labels)	Si	Labels Mermaid.
`threats_final`	No	Ya salen en espanol del Synthesizer.
`executive_summary`	No	Ya sale en espanol del Synthesizer.

Reglas de traduccion

Preservar estructura JSON, IDs, numeros, sintaxis Mermaid y terminos tecnicos (JWT, XSS, SQLi).

4.13 12. Report Generator (Fase V)

Archivo: `report_generator.py` **LLM:** ninguno

Rol

Genera CSV, Markdown y LaTeX de forma deterministica. Es el unico agente sin LLM.

Outputs generados

Output	Formato	Contenido
`csv_output`	CSV	Amenazas con STRIDE + DREAD + mitigaciones.
`report_output`	Markdown	Reporte con resumen, DFD, tablas, debate, arboles.
`latex_output`	LaTeX	Reporte profesional con longtable y color-coding.

Columnas CSV

Columnas del CSV de amenazas

```
ID Amenaza, Escenario de Amenaza, STRIDE, Control de Amenaza,
D, R, E, A, D.1, Puntaje DREAD, Prioridad, Estado,
Tratamiento de Riesgo, Jira Ticket, Justificacion/Observaciones
```

Estructura del reporte Markdown

1) Encabezado con nombre del sistema y fecha. 2) Resumen ejecutivo. 3) DFD Mermaid. 4) Tabla de amenazas. 5) Detalle por amenaza. 6) Analisis DREAD. 7) Historial de debate. 8) Arboles de ataque.

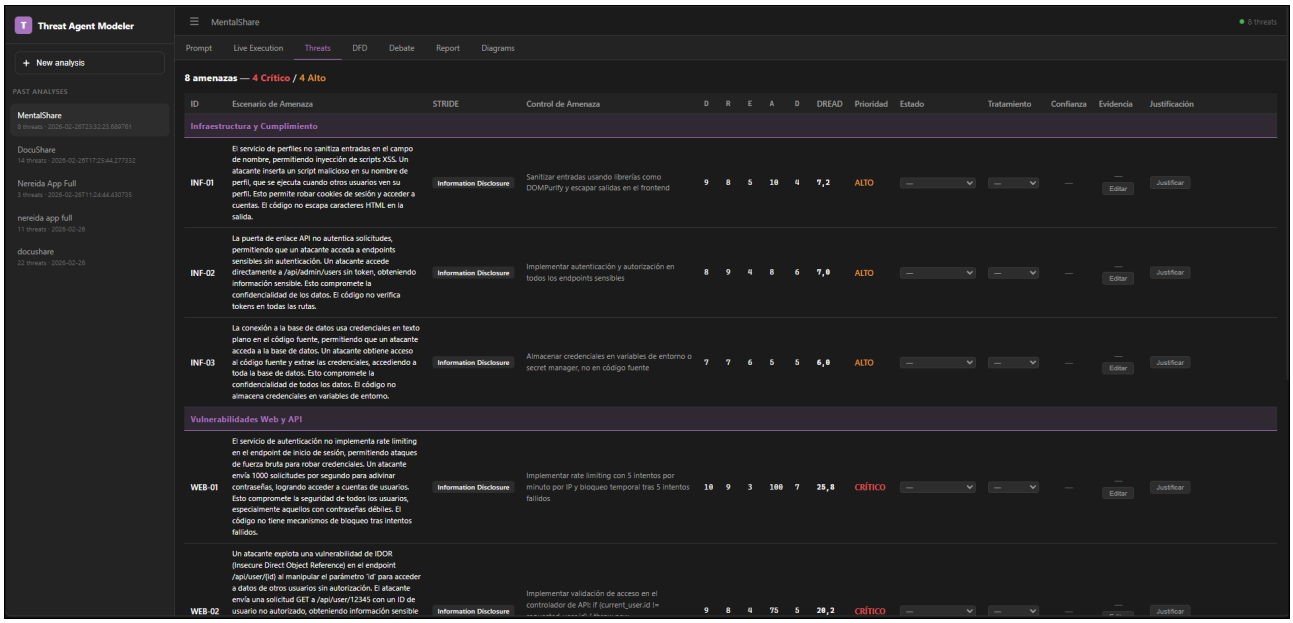


Figure 4.1: Ejemplo de tabla de amenazas con scoring DREAD.

Clasificacion de amenazas

Se agrupan por categorias del Synthesizer: infraestructura, privacidad, web/api, integracion agentica, IA/LLM, factores humanos y generales.

4.14 Resumen de herramientas RAG por agente

Tool set	Agentes que lo usan
ANALYST_TOOLS	STRIDE, PASTA, Attack Tree.
AI_ANALYST_TOOLS	MAESTRO, AI Threat.
DEBATE_TOOLS	Red Team, Blue Team.
SYNTHESIS_TOOLS	Threat Synthesizer.
VALIDATOR_TOOLS	DREAD Validator.
(ninguno)	Architecture Parser, Output Localizer, Report Generator.

Chapter 5

Sistema RAG

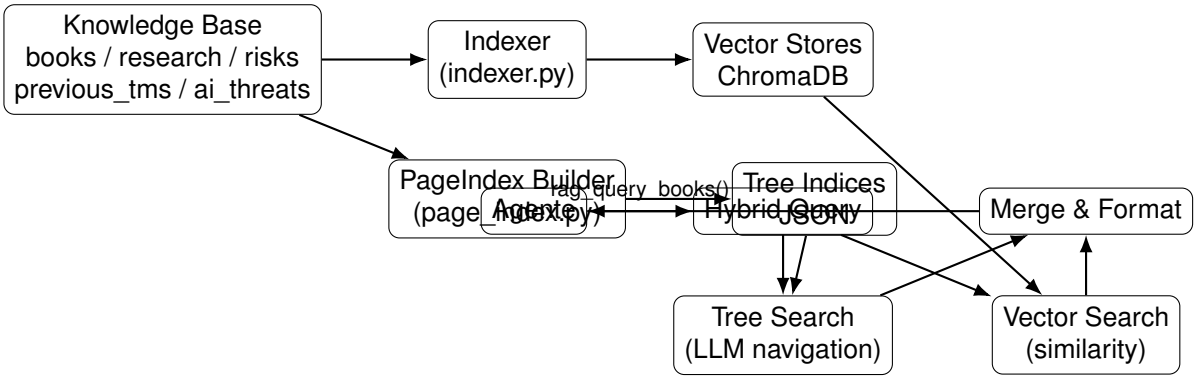
05 – Sistema RAG

Arquitectura dual ChromaDB + PageIndex: como AgenticTM consulta su knowledge base.

5.1 Vision General

El sistema RAG (Retrieval-Augmented Generation) combina dos estrategias de retrieval:

1. **Vector Search (ChromaDB)**: busqueda por similitud semantica sobre chunks.
2. **Tree Navigation (PageIndex)**: navegacion estructural por tabla de contenidos de PDFs.



5.2 Knowledge Base: Estructura

```
knowledge_base/
|-- books/              # 8 PDFs
|   |-- Threat_Modeling_Designing_for_Security.pdf
|   |-- OWASP_Testing_Guide.pdf
|   `-- ...
|-- research/           # 4 docs
|   |-- advanced_threat_modeling_techniques.md
|   `-- ...
|-- risks_mitigations/  # 1 CSV
|   `-- risks_controls_mapping.csv
|-- previous_threat_models/  # 17 CSVs
|   |-- Authentication_QR_Core.csv
|   |-- Chatbot_Infobip_Whatsapp.csv
|   |-- Enrolamiento_Biometrico.csv
```

```
|   `-- ...
`-- ai_threats/         # 7+ archivos
    |-- deck.json
    |-- owasp_llm_top10.pdf
    `-- ...
```

Vector stores

Store	Directorio	Tipos	Uso principal
`books`	`books/`	PDF	Libros tecnicos de referencia.
`research`	`research/`	MD, PDF	Papers y guias de threat modeling.
`risks_mitigations`	`risks_mitigations/`	CSV	Riesgos genericos y controles.
`previous_threat_models`	`previous_threat_models/`	CSV	TMs previos como few-shot.
`ai_threats`	`ai_threats/`	PDF, JSON, CSV	Amenazas AI/ML.

Embeddings

- Modelo: `nomic-embed-text` (via Ollama).
- Chunk size: 1000 caracteres.
- Chunk overlap: 200 caracteres.
- Splitter: `RecursiveCharacterTextSplitter`.
- Tamano del modelo: 274 MB.

5.3 ChromaDB: Busqueda Vectorial

Indexacion (`indexer.py`, 348 lineas)

Tipo	Procesamiento
PDF	`PyMuPDFLoader` -> paginas -> chunks.
CSV	Pandas -> headers + rows como chunks.
JSON	Deteccion PLOT4ai -> cards como chunks.
Markdown	Texto directo -> chunks.
TXT	Texto directo -> chunks.

Indexacion incremental

```
def _file_hash(path: Path) -> str:
    return hashlib.md5(path.read_bytes()).hexdigest()
if current_hash != stored_hash and not force:
    # skip
```

PLOT4ai deck parser

```
for card in deck["cards"]:
    content = f"Category: {card['category']}\n"
    content += f"Title: {card['title']}\n"
    content += f"Description: {card['description']}\n"
```

Consulta

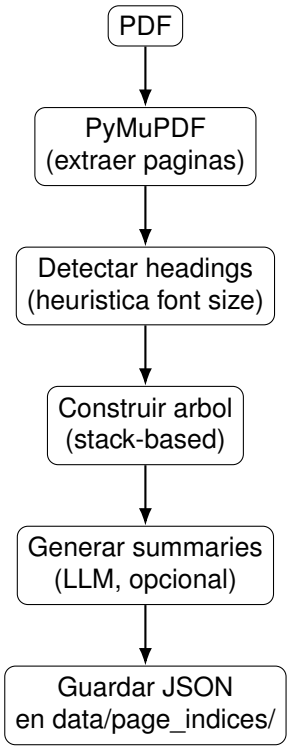
```
class RAGStoreManager:
    def query(self, store_name, query, top_k=5) -> str:
        store = self.get_store(store_name)
        results = store.similarity_search(query, k=top_k)
        return formatted_text
```

5.4 PageIndex: Navegacion Estructural

Concepto

Para PDFs largos, la busqueda vectorial por chunks puede perder contexto. PageIndex construye un arbol de tabla de contenidos y permite navegacion inteligente por secciones.

Construccion (`page_index.py`, 471 lineas)



Deteccion de headings

```
for block in page.get_text("dict")["blocks"]:
    for line in block["lines"]:
        for span in line["spans"]:
            if span["size"] > avg_size * 1.3:
                # heading
```

Estructura del arbol

```
{
  "doc_name": "Threat_Modeling_Designing_for_Security.pdf",
  "root": {
    "title": "Threat Modeling: Designing for Security",
    "level": 0,
    "page_range": [0, 320],
    "summary": "Comprehensive guide to threat modeling...",
    "children": [
      {
        "title": "Chapter 1: Introduction to Threat Modeling",
        "level": 1,
        "page_range": [1, 25],
        "summary": "Introduces core concepts...",
        "children": [
          {
            "title": "1.1 What is Threat Modeling?",
            "level": 2,
            "page_range": [2, 8],
            "text": "Threat modeling is a structured approach..."
          }
        ]
      }
    ]
  }
}
```

Summaries LLM (opcional)

Si `tree_summaries: true`, se usa el Quick Thinker para generar summaries de nodos (maximo `max_summary_nodes: 50` por PDF).

Retrieval (`tree_retriever.py`, 310 lineas)

1) LLM Tree Navigation

```
def llm_tree_search(query, trees, llm, top_k=3):
    # Navegacion guiada por LLM
```

2) Keyword fallback

```
def keyword_tree_search(query, trees, top_k=3):
    # Fallback sin LLM, scoring simplificado
```

Hybrid merge

```
def hybrid_search(query, trees, vector_results, llm, top_k=5):
    # Combina tree + vector con deduplicacion 70%
```

5.5 Herramientas RAG (`tools.py`, 92 lineas)

Los agentes acceden al RAG via herramientas LangChain decoradas con `@tool`:

```
@tool
def rag_query_books(query: str) -> str:
    """Consulta libros (PDFs) con busqueda hibrida."""

@tool
def rag_query_research(query: str) -> str:
    """Consulta papers y guias."""

@tool
def rag_query_risks(query: str) -> str:
    """Consulta riesgos y mitigaciones."""

@tool
def rag_query_previous_tms(query: str) -> str:
    """Consulta threat models previos."""

@tool
def rag_query_ai_threats(query: str) -> str:
    """Consulta amenazas AI/ML."""
```

Instruccion de citacion

```
_CITATION_INSTRUCTION = (
    "IMPORTANT: When using facts from the above sources, "
    "include them in the threat's `evidence_sources` field:\n"
    '  {"source_type": "rag", "source_name": "<doc>", "excerpt": "<quote>"}'
)
```

Tool sets por fase

Set	Agentes
ANALYST_TOOLS	STRIDE, PASTA, Attack Tree.
AI_ANALYST_TOOLS	MAESTRO, AI Threat.
DEBATE_TOOLS	Red Team, Blue Team.
SYNTHESIS_TOOLS	Threat Synthesizer.
VALIDATOR_TOOLS	DREAD Validator.
ALL_RAG_TOOLS	Uso general.

5.6 Categorias (`categories.py`, 151 lineas)

Auto-deteccion

```
CATEGORY_KEYWORDS = {
  "aws": ["aws", "amazon", "ec2", "s3", "lambda", "rds", "dynamodb", ...],
  "azure": ["azure", "microsoft", "entra", "aks", "app service", ...],
  "gcp": ["gcp", "google cloud", "compute engine", "cloud run", ...],
  "ai": ["ai", "ml", "llm", "model", "agent", "rag", "embedding", ...],
  "mobile": ["mobile", "ios", "android", "react native", "flutter", ...],
  "web": ["web", "browser", "frontend", "react", "angular", "xss", ...],
  "iot": ["iot", "sensor", "firmware", "mqtt", "zigbee", "bluetooth", ...],
  "privacy": ["privacy", "gdpr", "hipaa", "pii", "consent", ...],
  "supply_chain": ["supply chain", "third-party", "vendor", "sbom", ...],
```

```
}
```

Scoring con threshold

```
def detect_categories(system_description: str) -> list[str]:
    for category, keywords in CATEGORY_KEYWORDS.items():
        hits = sum(1 for kw in keywords if kw in text)
        if hits >= 2:
            detected.add(category)
    return sorted(detected | {"base"})
```

Filtrado en consulta

```
@tool
def rag_query_risks(query: str) -> str:
    raw = manager.query("risks_mitigations", query, top_k=8)
    if active_categories:
        sections = raw.split("---")
        filtered = [s for s in sections if matches_categories(s)]
        return filtered_text
    return raw
```

Ejemplo practico

Si el input describe un sistema AWS con componentes AI:

1. `detect_categories()` detecta `["aws", "ai", "base"]`.
2. `set_active_categories(["aws", "ai", "base"])` configura el filtro.
3. `rag_query_risks("authentication bypass")` retorna riesgos relevantes.

5.7 `RAGStoreManager` (175 lineas)

Inicializacion

```
class RAGStoreManager:
    def __init__(self, persist_dir, embedding_model, embedding_provider,
                 base_url, tree_index_dir):
        self._embeddings = OllamaEmbeddings(
            model=embedding_model,
            base_url=base_url,
        )
        self._stores = {}
        self._trees = {}
        self._tree_llm = None
```

Hybrid query

```
def hybrid_query(self, store_name, query, top_k=5):
    vector_text = self.query(store_name, query, top_k)
    if store_name in ["books", "research", "ai_threats"] and self._trees:
        tree_results = self._tree_search(query, top_k=3)
        if tree_results:
```

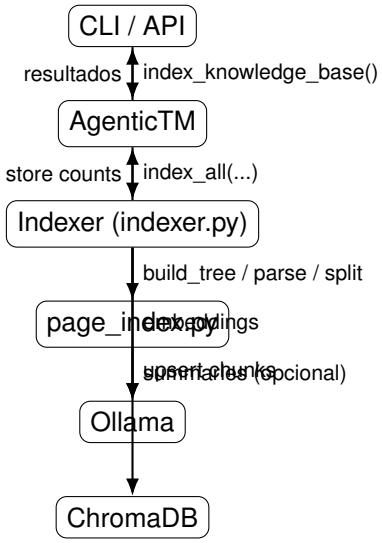
```
            tree_text = self._format_tree_results(tree_results)
            return f"=== Tree-based results ===\n{tree_text}\n\n=== Vector-based
            ↪  results ===\n{vector_text}"
    return vector_text
```

5.8 Indexacion: Flujo Completo

Comando CLI

```
python cli.py index
python cli.py index --verbose
python cli.py index --path ruta/a/mi/knowledge_base
```

Lo que sucede



Resultado de ejemplo

```
AgenticTM -- RAG Indexing
Knowledge Base: knowledge_base

Store                    Chunks
books                    1247
research                  156
risks_mitigations           89
previous_threat_models     423
ai_threats                 312
TOTAL                     2227
```

5.9 Datos: 14 Tree Indices

Los indices de arbol se guardan en `data/page_indices/`:

```
data/page_indices/
|-- threat_modeling_designing_for_security.json
|-- owasp_testing_guide.json
|-- nist_sp_800_53.json
```

```
|-- ... (uno por cada PDF en books/, research/, ai_threats/)
`-- total: 14 archivos
```

Cada archivo pesa entre 50 KB y 2 MB.

5.10 Limitaciones Conocidas

Limitacion	Impacto	Mejora propuesta
Sin reranking	Resultados menos relevantes	Agregar cross-encoder reranker.
Chunks fijos de 1000 chars	Puede cortar contexto	Semantic chunking por headers.
Sin evaluacion de calidad RAG	No se sabe utilidad de chunks	Metricas de retrieval.
Sin citacion estructurada	Fuentes no siempre preservadas	Forzar citacion estructurada.
Keyword filtering	Falsos positivos/negativos	Clasificacion ML.

Chapter 6

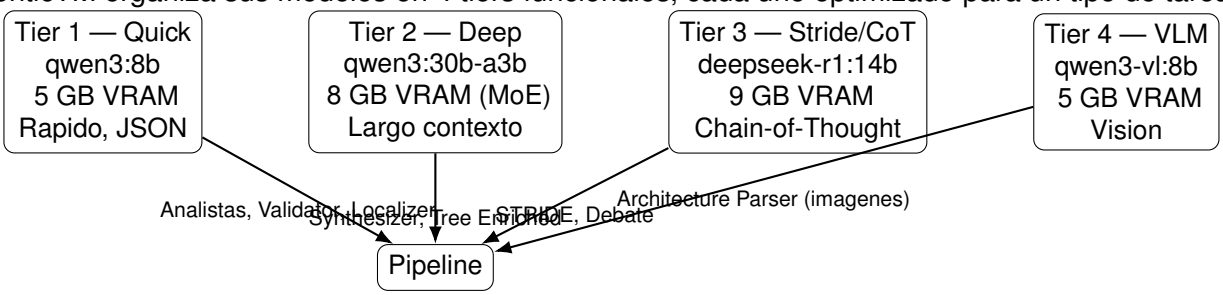
Modelos LLM

06 – Modelos LLM

Arquitectura de 4 tiers, LLMFactory, 5 providers y recomendaciones de hardware.

6.1 Arquitectura de 4 Tiers

AgenticTM organiza sus modelos en 4 tiers funcionales, cada uno optimizado para un tipo de tarea.



Detalle por tier

Tier	Config key	Modelo	Parametros	VRAM	Temp	Proposito
Quick	`quick_thinker`	`qwen3:8b`	8B densas	5 GB	0.3	Triage rapido, JSON estructurado.
Deep	`deep_thinker`	`qwen3:30b-a3b`	30B MoE (3.3B activos)	8 GB	0.2	Sintesis compleja, contexto largo.
Stride/CoT	`stride_thinker`	`deepseek-r1:14b`	14B densas	9 GB	0.3	CoT visible, audit trail.
VLM	`vlm`	`qwen3-vl:8b`	8B + vision	5 GB	0.1	Analisis de diagramas e imagenes.

Por que `qwen3:30b-a3b` para Deep?

Usa Mixture of Experts (MoE): 30B totales pero 3.3B activos por paso. Esto combina capacidad alta, velocidad cercana a modelos pequenos y VRAM alrededor de 8 GB. Es ideal para el Synthesizer.

Por que DeepSeek-R1 para STRIDE y debate?

DeepSeek-R1 es CoT nativo. Produce audit trail visible para STRIDE y transparencia en el debate. `_strip_think_tags()` elimina los tags `<think>...</think>` del output final.

6.2 LLMFactory (`agentictm/llm/__init__.py`, 165 lineas)

Propiedades cacheadas

```
class LLMFactory:
    @property
    def quick(self) -> BaseChatModel:
        """LLM rapido para analistas (free-text)."""
```

```
    @property
    def quick_json(self) -> BaseChatModel:
        """LLM rapido con format=json."""
    @property
    def deep(self) -> BaseChatModel:
        """LLM deep para Synthesizer (free-text)."""
    @property
    def deep_json(self) -> BaseChatModel:
        """LLM deep con format=json."""
    @property
    def stride(self) -> BaseChatModel:
        """LLM CoT para STRIDE/debate (free-text)."""
    @property
    def stride_json(self) -> BaseChatModel:
        """LLM CoT con format=json."""
    @property
    def vlm(self) -> BaseChatModel:
        """Vision Language Model."""
```

Cada property usa `_get_or_create(key, cfg, format_override)` para cachear instancias.

`format_override="json"`

Los tiers `*_json` pasan `format="json"` a Ollama para forzar salida JSON valida.

6.3 5 Providers Soportados

Provider	Import	Config
Ollama (default)	`langchain_ollama.ChatOllama`	`provider: "ollama", base_url: "http://localhost:11434"`
Anthropic	`langchain_anthropic.ChatAnthropic`	`provider: "anthropic", api_key: "sk-..."`
Google	`langchain_google_genai.ChatGoogleGenerativeAI`	`provider: "google", api_key: "AIza..."`
OpenAI	`langchain_openai.ChatOpenAI`	`provider: "openai", api_key: "sk-..."`
Azure OpenAI	`langchain_openai.AzureChatOpenAI`	`provider: "azure", base_url: "https://xxx.openai.azure.com"`

Instalacion de providers cloud

```
pip install .
pip install ".[cloud]"
```

Ejemplo: configuracion hibrida

```
{
  "quick_thinker": {
```

```
    "provider": "ollama",
    "model": "qwen3:8b",
    "base_url": "http://localhost:11434"
  },
  "deep_thinker": {
    "provider": "anthropic",
    "model": "claude-sonnet-4-20250514",
    "api_key": "sk-ant-..."
  },
  "stride_thinker": {
    "provider": "ollama",
    "model": "deepseek-r1:14b",
    "base_url": "http://localhost:11434"
  },
  "vlm": {
    "provider": "google",
    "model": "gemini-2.0-flash",
    "api_key": "AIza..."
  }
}
```

6.4 Modelos Ollama Requeridos

Descarga

```
ollama pull qwen3:8b
ollama pull qwen3:30b-a3b
ollama pull deepseek-r1:14b
ollama pull qwen3-vl:8b
ollama pull nomic-embed-text
```

Total aproximado: 37 GB de disco.

Verificacion

```
ollama list
ollama run qwen3:8b "Hola, funciona?"
```

Control de GPU

```
{
  "deep_thinker": { "model": "qwen3:30b-a3b", "num_gpu": -1 },
  "quick_thinker": { "model": "qwen3:8b", "num_gpu": null }
}
```

Valor	Significado
`null`	Ollama decide automaticamente (default).
`-1`	Todas las capas en GPU (max velocidad).
`0`	Todas las capas en CPU (sin GPU).
`N > 0`	N capas en GPU (resto en CPU).

6.5 Recomendaciones de Hardware

GPU por presupuesto

VRAM	Configuracion recomendada
8 GB	quick=qwen3:8b, deep=qwen3:8b, `max_parallel_analysts=1`, `cascade`.
16 GB	quick=qwen3:8b, deep=qwen3:30b-a3b, `max_parallel_analysts=2`, `hybrid`.
24 GB	igual a 16 GB con `max_parallel_analysts=3`.
32+ GB	`parallel` con `max_parallel_analysts=5`.

Consumo estimado por fase

Fase	Modelos activos	VRAM pico (hybrid)
I	quick_json + vlm (si hay imagenes)	10 GB
II	2x quick_json (throttled)	10 GB
III	stride	9 GB
II.5	deep_json	8 GB
IV	deep_json -> quick_json	8 GB -> 5 GB
V	quick_json (localizer)	5 GB

Configuraciones probadas

Hardware	Tiempo tipico	Config
RTX 4090 (24 GB)	15–25 min	hybrid, max=2, todos los modelos.
RTX 3080 (10 GB)	25–40 min	hybrid, max=1, quick=deep=qwen3:8b.
Apple M2 Ultra (64 GB)	10–20 min	hybrid, max=3, todos los modelos.
CPU only (32 GB RAM)	60–120 min	cascade, quick=deep=qwen3:4b.

6.6 Timeouts

Configuracion	Default	Descripcion
`timeout` (por LLM)	300s (quick), 600s (deep/stride/vlm)	HTTP client timeout.
`vlm_image_timeout`	600s (quick), 1200s (vlm)	Timeout por imagen.

Si un modelo excede el timeout, la request falla y `_safe_node` maneja el error.

Chapter 7

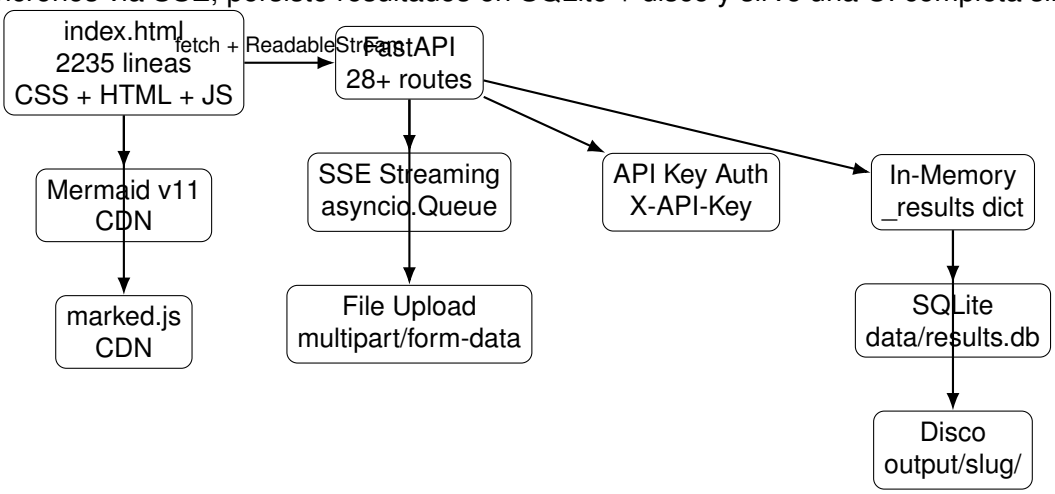
API y Frontend

07 – API y Frontend

FastAPI backend con SSE streaming, SPA frontend con dark theme, Mermaid y 7 tabs.

7.1 Vision General

AgenticTM expone una API REST (FastAPI) y una SPA embebida. El servidor maneja analisis asincronos via SSE, persiste resultados en SQLite + disco y sirve una UI completa sin build externo.



Estructura del paquete

Archivo	Lineas	Descripcion
`agentictm/api/server.py`	1709	FastAPI: rutas, SSE, uploads, auth, CRUD.
`agentictm/api/storage.py`	157	`ResultStore` con `aiosqlite`.
`agentictm/api/__init__.py`	2	Docstring del backend.
`agentictm/api/static/index.html`	2235	SPA completa (CSS/HTML/JS).

7.2 Rutas API — Referencia Completa

Health

Metodo	Path	Descripcion	Auth
GET	`/api/health`	Liveness probe.	No
GET	`/api/ready`	Readiness probe (verifica Ollama via httpx).	No

Analysis

Metodo	Path	Descripcion	Auth
POST	/api/analyze	Inicia analisis (SSE).	Si
POST	/api/upload	Sube archivo para analisis.	Si
GET	/api/categories	Lista categorias disponibles.	No
GET	/api/analysis/\{id\}/status	Estado de analisis activo.	No
GET	/api/analyses/active	Lista analisis activos/en cola.	No

Results

Metodo	Path	Descripcion	Auth
GET	/api/results	Lista analisis completados.	No
GET	/api/results/\{id\}	Resultado completo.	No
DELETE	/api/results/\{id\}	Elimina analisis.	Si
GET	/api/results/\{id\}/csv	Descarga CSV.	No
GET	/api/results/\{id\}/csv-justified	CSV con justificacion.	No
GET	/api/results/\{id\}/report	Reporte Markdown.	No
GET	/api/results/\{id\}/latex	Reporte LaTeX.	No
GET	/api/results/\{id\}/attachments/\{name\}	Descarga adjunto.	No
PUT	/api/results/\{id\}/threats/\{tid\}/justify	Agrega justificacion.	Si
DELETE	/api/results/\{id\}/threats/\{tid\}/justify	Elimina justificacion.	Si
PUT	/api/results/\{id\}/threats/\{tid\}/field	Actualiza campo.	Si

Knowledge Base

Metodo	Path	Descripcion	Auth
GET	/api/knowledge-base	Lista documentos KB.	No
POST	/api/knowledge-base/upload	Sube doc a KB.	Si
DELETE	/api/knowledge-base/\{cat\}/\{file\}	Elimina doc.	Si
POST	/api/knowledge-base/reindex	Re-indexacion.	Si
GET	/api/knowledge-base/categories	Categorias KB.	No

Observability

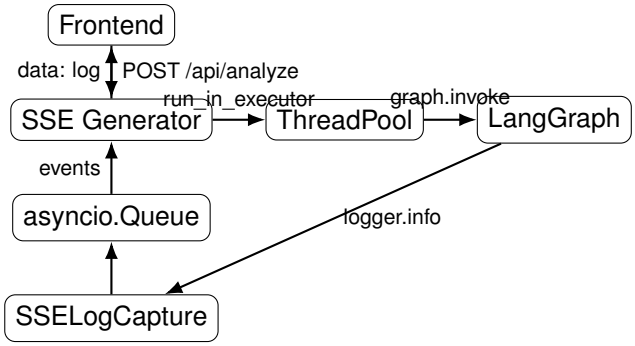
Metodo	Path	Descripcion	Auth
GET	/api/metrics	Dashboard global.	No
GET	/api/results/\{id\}/metrics	Metricas por analisis.	No

SPA Routes

Path	Descripcion
/	Root, sirve SPA.
/home, /prompt, /live, /threats, /dfd, /debate, /report, /diagrams	Tabs.
/\{project_slug\}/\{tab\}	Rutas con scope de proyecto.
/static/*	Archivos estaticos.

7.3 SSE Streaming — Implementacion

Puente queue-based



Tipos de eventos SSE

Tipo	Campos	Cuando
start	analysis_id, system_name, categories, timestamp	Inicio de analisis.
log	level, message, agent, timestamp	Cada log del pipeline.
heartbeat	type	Cada 500ms sin actividad.
complete	analysis_id, threats_count, categories	Pipeline exitoso.
error	message, traceback, timestamp	Excepcion en pipeline.

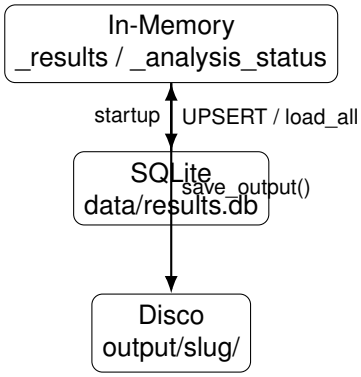
Headers SSE

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
X-Accel-Buffering: no
```

7.4 Autenticacion

Aspecto	Detalle
Header	`X-API-Key`.
Dependency	`verify_api_key()` con `Depends()`.
Sin key	Auth deshabilitada.
Con key	Header debe coincidir, devuelve 401.
Rutas protegidas	Analyze, upload, delete results, justify, KB upload/delete/reindex.
Rutas publicas	Health, categories, GET results, descargas.

7.5 Persistencia — 3 Capas



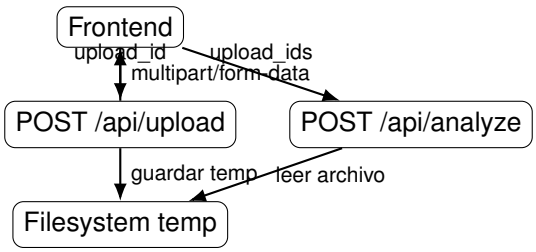
SQLite — schema

```
CREATE TABLE results (
    analysis_id   TEXT PRIMARY KEY,
    system_name   TEXT NOT NULL DEFAULT '',
    created_at    TEXT NOT NULL,
    updated_at    TEXT NOT NULL,
    output_dir    TEXT DEFAULT '',
    result_json   TEXT NOT NULL
);
CREATE INDEX idx_results_created ON results(created_at DESC);
```

Operaciones ResultStore

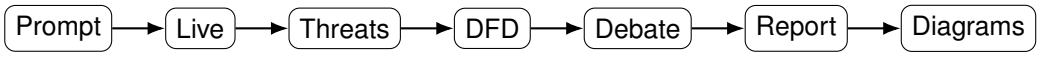
Metodo	Descripcion
`save()`	UPSERT (reescribe JSON).
`get(id)`	Un resultado por ID.
`list_all()`	Metadata para listados.
`list_full()`	Todo, usado en startup.
`delete(id)`	Elimina registro.
`count()`	Total de analisis.

7.6 Manejo de Uploads

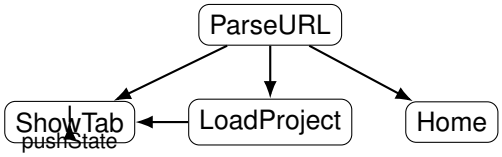


7.7 Frontend SPA

7 tabs



Routing



7.8 Dependencias CDN

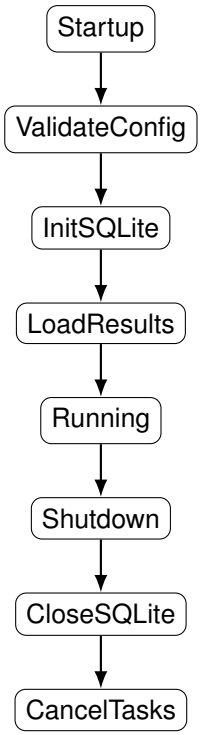
Libreria	Version	Uso
Mermaid	v11	Diagramas DFD y Attack Trees.
marked.js	Latest	Render Markdown.

7.9 Lifespan y middleware

CORS

```
allow_origins=["*"]
allow_methods=["*"]
allow_headers=["*"]
allow_credentials=True
```

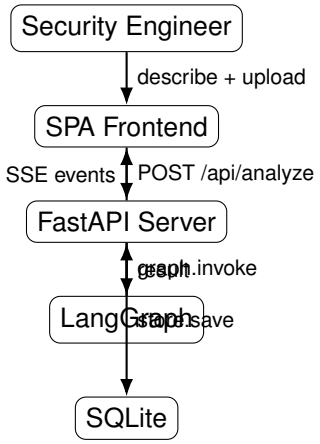
Lifespan



7.10 Metricas de Observabilidad

```
GET /api/metrics
GET /api/results/{id}/metrics
```

7.11 Ejemplo: Flujo Completo de Analisis



Chapter 8

Configuracion

08 – Configuracion

Referencia completa de `config.json`, variables de entorno y modelos Pydantic.

8.1 Archivo `config.json`

El archivo `config.json` controla toda la configuracion de AgenticTM. Se carga al inicio via `AgenticTMConfig.load(`

Estructura completa (con defaults)

```
{
  "quick_thinker": {
    "provider": "ollama",
    "model": "qwen3:8b",
    "temperature": 0.3,
    "base_url": "http://localhost:11434",
    "api_key": null,
    "timeout": 300,
    "format": null,
    "num_gpu": null,
    "vlm_image_timeout": 600
  },
  "deep_thinker": {
    "provider": "ollama",
    "model": "qwen3:30b-a3b",
    "temperature": 0.2,
    "base_url": "http://localhost:11434",
    "api_key": null,
    "timeout": 600,
    "num_gpu": -1
  },
  "stride_thinker": {
    "provider": "ollama",
    "model": "deepseek-r1:14b",
    "temperature": 0.3,
    "base_url": "http://localhost:11434",
    "api_key": null,
    "timeout": 600,
    "num_gpu": -1
  },
  "vlm": {
    "provider": "ollama",
```

```
    "model": "qwen3-vl:8b",
    "temperature": 0.1,
    "base_url": "http://localhost:11434",
    "api_key": null,
    "timeout": 600,
    "vlm_image_timeout": 1200,
    "num_gpu": -1
  },
  "rag": {
    "knowledge_base_path": "knowledge_base",
    "vector_store_path": "data/vector_stores",
    "page_index_path": "data/page_indices",
    "embedding_provider": "ollama",
    "embedding_model": "nomic-embed-text",
    "chunk_size": 1000,
    "chunk_overlap": 200,
    "retrieval_top_k": 5,
    "tree_summaries": true,
    "max_summary_nodes": 50
  },
  "pipeline": {
    "max_debate_rounds": 2,
    "max_validation_iterations": 2,
    "enable_maestro": true,
    "output_format": "both",
    "output_language": "es",
    "csv_schema": "auto",
    "self_reflection_enabled": false,
    "self_reflection_rounds": 1,
    "min_threats": 8,
    "max_threats": 40,
    "target_threats": 20,
    "threat_categories": ["auto"],
    "analyst_execution_mode": "hybrid",
    "max_parallel_analysts": 2
  },
  "memory": {
    "enabled": true,
    "db_path": "data/memory.db",
    "journal_path": "data/journal.db"
  },
  "security": {
    "api_key": null,
    "max_input_length": 100000,
    "max_upload_size_mb": 10,
    "allowed_extensions": [
      ".txt", ".md", ".pdf", ".doc", ".docx", ".csv", ".json",
      ".yaml", ".yml", ".png", ".jpg", ".jpeg", ".gif",
      ".bmp", ".webp", ".svg", ".tiff"
    ]
  },
  "output_dir": "output"
}
```

8.2 Referencia por seccion

LLMConfig

Campo	Tipo	Default	Descripcion
provider	str	"ollama"	Provider LLM.
model	str	segun tier	Nombre del modelo.
temperature	float	0.3/0.2/0.1	Sampling (0.0 deterministico).
base_url	str	"http://localhost:11434"	URL base del provider.
api_key	str|None	null	API key cloud.
timeout	int	300/600	HTTP client timeout.
format	str|None	null	"json" para forzar JSON.
num_gpu	int|None	null o -1	Capas en GPU.
vlm_image_timeout	int	600/1200	Timeout por imagen.

RAGConfig

Campo	Tipo	Default	Descripcion
knowledge_base_path	Path	knowledge_base	Documentos KB.
vector_store_path	Path	data/vector_stores	ChromaDB.
page_index_path	Path	data/page_indices	Tree indices.
embedding_provider	str	"ollama"	Provider embeddings.
embedding_model	str	"nomic-embed-text"	Modelo embeddings.
chunk_size	int	1000	Tamano de chunks.
chunk_overlap	int	200	Overlap.
retrieval_top_k	int	5	Resultados por query.
tree_summaries	bool	true	Summaries por nodo.
max_summary_nodes	int	50	Max nodos por PDF.

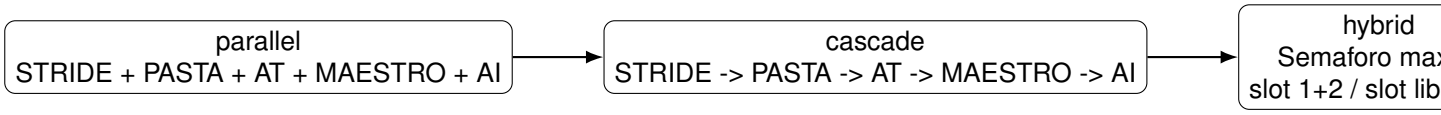
PipelineConfig

Campo	Tipo	Default	Descripcion
max_debate_rounds	int	2	Max rounds.
max_validation_iterations	int	2	Iteraciones DREAD.
enable_maestro	bool	true	Habilita MAESTRO.
output_format	str	"both"	csv/markdown/both.
output_language	str	"es"	es/en.
csv_schema	str	"auto"	auto/default.
self_reflection_enabled	bool	false	Critique -> revise.
self_reflection_rounds	int	1	Ciclos.
min_threats	int	8	Min amenazas.
max_threats	int	40	Max amenazas.
target_threats	int	20	Target prompt.
threat_categories	list[str]	["auto"]	Categorias activas.
analyst_execution_mode	str	"hybrid"	parallel/cascade/hybrid.
max_parallel_analysts	int	2	LLM calls simultaneas.

Categorias disponibles

Categoria	Descripcion	KB Directory
`"auto"`	Auto-detectar del input.	–
`"base"`	Siempre incluida.	`risks_mitigations/base/`
`"aws"`	Amenazas AWS.	`risks_mitigations/aws/`
`"azure"`	Amenazas Azure.	`risks_mitigations/azure/`
`"gcp"`	Amenazas GCP.	`risks_mitigations/gcp/`
`"ai"`	AI/ML threats.	`risks_mitigations/ai/`
`"mobile"`	Apps moviles.	`risks_mitigations/mobile/`
`"web"`	Amenazas web.	`risks_mitigations/web/`
`"iot"`	IoT/embedded.	`risks_mitigations/iot/`
`"privacy"`	Privacidad.	`risks_mitigations/privacy/`
`"supply_chain"`	Supply chain.	`risks_mitigations/supply_chain/`

Modos de ejecucion de analistas



MemoryConfig

Campo	Tipo	Default	Descripcion
`enabled`	`bool`	`true`	Memoria persistente.
`db_path`	`Path`	`data/memory.db`	SQLite memoria.
`journal_path`	`Path`	`data/journal.db`	SQLite journal.

SecurityConfig

Campo	Tipo	Default	Descripcion
`api_key`	`str|None`	`null`	API key.
`max_input_length`	`int`	100000	Max chars input.
`max_upload_size_mb`	`int`	10	Max upload MB.
`allowed_extensions`	`list[str]`	17 extensiones	Extensiones permitidas.

8.3 Variables de entorno

Las variables de entorno tienen precedencia sobre `config.json`.

Variable	Seccion
`AGENTICTM_API_KEY`	`security.api`
`AGENTICTM_OLLAMA_URL`	`*.base_url`
`AGENTICTM_OUTPUT_DIR`	`output_dir`
`AGENTICTM_MAX_INPUT_LENGTH`	`security.max`
`AGENTICTM_MAX_UPLOAD_MB`	`security.max`

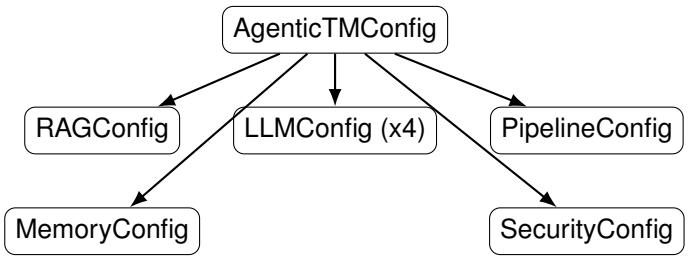
Ejemplo

```
set AGENTICTM_API_KEY=mi-clave-secreta-123
```

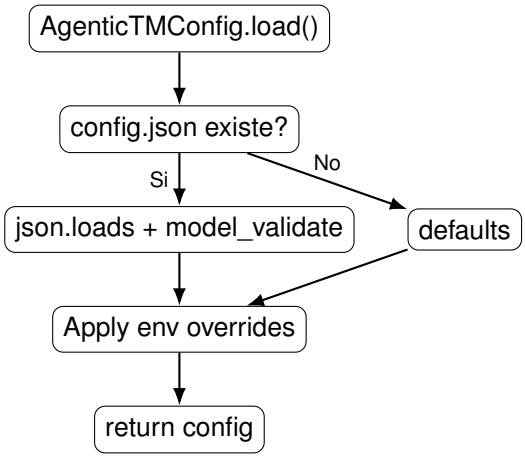
```
set AGENTICTM_OLLAMA_URL=http://192.168.1.100:11434
set AGENTICTM_OUTPUT_DIR=D:\resultados
```

8.4 Modelos Pydantic

Jerarquia de clases



Flujo de carga



8.5 `validate_startup()` — validacion al inicio

Retorna warnings (no fatales) y crea directorios faltantes.

Check	Accion si falla
`knowledge_base_path`	Crea directorio.
`vector_store_path`	Crea directorio.
`page_index_path`	Crea directorio.
`output_dir`	Crea directorio.
Ollama accesible	Warning en log.
Modelos disponibles	Warning con lista.
`security.api_key`	Warning: endpoints sin auth.

Ejemplo de output

```
Ollama connected at http://localhost:11434 (5 models available)
quick_thinker: qwen3:8b
deep_thinker: qwen3:30b-a3b
stride_thinker: qwen3:8b
vlm: qwen3-vl:8b NOT FOUND
No API key configured -- endpoints are unprotected
5 vector stores found
```

```
14 tree indices found
```

8.6 Configuracion para Docker

```yaml
services:
  app:
    environment:
      - AGENTICTM_OLLAMA_URL=http://ollama:11434
      - AGENTICTM_API_KEY=${API_KEY:-}
      - AGENTICTM_OUTPUT_DIR=app/output
    volumes:
      - ./config.json:app/config.json:ro
      - ./output:app/output
      - ./knowledge_base:app/knowledge_base:ro
```

8.7 Configuraciones recomendadas

Computadora personal (16 GB VRAM)

```json
{
  "quick_thinker": { "model": "qwen3:8b" },
  "deep_thinker": { "model": "qwen3:30b-a3b", "num_gpu": -1 },
  "stride_thinker": { "model": "qwen3:8b" },
  "vlm": { "model": "qwen3-vl:8b" },
  "pipeline": {
    "analyst_execution_mode": "hybrid",
    "max_parallel_analysts": 2,
    "max_debate_rounds": 4
  }
}
```

Servidor GPU (32+ GB VRAM)

```json
{
  "quick_thinker": { "model": "qwen3:8b" },
  "deep_thinker": { "model": "qwen3:30b-a3b", "num_gpu": -1 },
  "stride_thinker": { "model": "deepseek-r1:14b", "num_gpu": -1 },
  "vlm": { "model": "qwen3-vl:8b", "num_gpu": -1 },
  "pipeline": {
    "analyst_execution_mode": "parallel",
    "max_parallel_analysts": 5,
    "max_debate_rounds": 8
  }
}
```

Cloud hibrido (GPU local + APIs)

```
{
  "quick_thinker": { "provider": "ollama", "model": "qwen3:8b" },
  "deep_thinker": { "provider": "anthropic", "model": "claude-sonnet-4-20250514",
  ↪  "api_key": "sk-ant-..." },
  "stride_thinker": { "provider": "ollama", "model": "deepseek-r1:14b" },
```

```
  "vlm": { "provider": "google", "model": "gemini-2.0-flash", "api_key": "AIza..." },
  "pipeline": {
    "analyst_execution_mode": "parallel",
    "max_parallel_analysts": 5
  }
}
```

Solo CPU

```
{
  "quick_thinker": { "model": "qwen3:4b", "num_gpu": 0 },
  "deep_thinker": { "model": "qwen3:8b", "num_gpu": 0 },
  "stride_thinker": { "model": "qwen3:4b", "num_gpu": 0 },
  "vlm": { "model": "qwen3-vl:4b", "num_gpu": 0 },
  "pipeline": {
    "analyst_execution_mode": "cascade",
    "max_parallel_analysts": 1,
    "max_debate_rounds": 2
  }
}
```

Chapter 9

Guia de Uso

09 – Guia de Uso

Instalacion paso a paso, CLI, interfaz web, Docker y flujo completo de analisis.

9.1 Requisitos Previos

Hardware minimo

Componente	Minimo	Recomendado
RAM	16 GB	32 GB
GPU VRAM	8 GB (opciones limitadas)	16–24 GB (NVIDIA)
Disco	50 GB libres	100 GB (modelos + KB)
CPU	4 cores	8+ cores

Software

Dependencia	Version	URL
Python	3.11+	`python.org`
Ollama	0.3.0+	`ollama.com`
Git	Cualquiera	`git-scm.com`
Docker (opcional)	24.0+	`docker.com`

9.2 Instalacion

1. Clonar el repositorio

```
git clone https://github.com/your-org/AgenticTM.git
cd AgenticTM
```

2. Crear entorno virtual

```
# Windows
python -m venv .venv
.venv\Scripts\activate

# Linux / macOS
python3 -m venv .venv
source .venv/bin/activate
```

3. Instalar dependencias

```
pip install -e .
pip install -r requirements.txt
```

4. Instalar Ollama y modelos

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen3:8b
ollama pull qwen3:30b-a3b
ollama pull deepseek-r1:14b
ollama pull qwen3-vl:8b
ollama pull nomic-embed-text
ollama list
```

5. Inicializar estructura

```
python cli.py init
```

```
knowledge_base/
|-- books/
|-- research/
|-- risks_mitigations/
|-- previous_threat_models/
`-- ai_threats/
data/
|-- vector_stores/
output/
config.json
```

6. Indexar knowledge base (opcional)

```
python cli.py index
```

Directorio	Que poner
`books/`	PDFs de libros de seguridad.
`research/`	Papers academicos, RFCs de seguridad.
`risks_mitigations/`	CAPEC/CWE/NIST por categoria.
`previous_threat_models/`	CSV de threat models previos.
`ai_threats/`	`deck.json` y papers de AI security.

9.3 Uso por CLI

Comando `analyze`

```
python cli.py analyze --name "Mi Sistema" --file descripcion.md
python cli.py analyze --name "API REST" --input "Sistema REST con JWT..."
python cli.py analyze --name "AWS App" --file desc.md --categories aws,ai,web
python cli.py analyze --file desc.md --config custom_config.json --verbose
```

Flags de `analyze`

Flag	Short	Default	Descripcion
`-input`	`-i`	—	Texto directo.
`-file`	`-f`	—	Archivo con descripcion.
`-name`	`-n`	`"System"`	Nombre del sistema.
`-categories`	`-cats`	`"auto"`	Categorias comma-separated.
`-output`	`-o`	`output/`	Directorio salida.
`-config`	`-c`	`config.json`	Archivo de configuracion.
`-verbose`	`-v`	`false`	Logging detallado.

Output del CLI

```
AgenticTM -- Threat Modeling
Sistema: DocuShare
Input: 2847 caracteres
Categorias: auto

Threat Model -- DocuShare
ID     | Component   | Threat        | DREAD | P
WEB-01 | API Gateway | SQL Injection |  38   | C
LLM-01 | AI Agent    | Prompt Inject |  35   | H

Output saved to: output/docushare-24-02-2026-2130/
```

Comando `index`

```
python cli.py index
python cli.py index --path ./my_knowledge_base/
python cli.py index --verbose
```

Flag	Short	Default	Descripcion
`-path`	`-p`	`knowledge_base`	Path KB.
`-config`	`-c`	`config.json`	Archivo de configuracion.
`-verbose`	`-v`	`false`	Logging detallado.

Comando `init`

```
python cli.py init
python cli.py init --path ./nuevo_proyecto/
```

9.4 Uso por interfaz web

1. Iniciar el server

```
python run.py
python run.py --port 8001
python run.py --reload
```

Abrir en el navegador: `http://localhost:8000`.

2. Pantalla de inicio (Home)

Textarea, drag & drop, selector de categorias, control de rounds y boton Run.

3. Usar cloud providers

Panel `API Keys`: pegar key, activar toggle, solo un provider a la vez. Keys en localStorage.

4. Escribir la descripcion del sistema

Inclui componentes, flujos, autenticacion y trust boundaries. Si usas AI/ML, mencionarlo.

5. Subir archivos

Formatos soportados: .png, .jpg, .svg, .webp, .txt, .md, .csv, .json, .yaml, .pdf.

6. Seleccionar categorias

Auto o especificas segun stack (aws, ai, web, mobile, privacy, etc.).

7. Monitorear ejecucion (Live)

Pipeline bar, thinking stream, auto-scroll. Tiempo tipico: 15–40 min.

8. Revisar resultados

Threats, DFD, Debate, Report y Diagrams.

9. Descargar outputs

CSV, CSV justified, Markdown, LaTeX.

9.5 Docker

Con docker-compose

```
docker-compose up -d
docker-compose logs -f app
```

docker-compose.yml

```
services:
  ollama:
    image: ollama/ollama:latest
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]
  app:
    build: .
```

```
    ports:
      - "8000:8000"
    environment:
      - AGENTICTM_OLLAMA_URL=http://ollama:11434
    volumes:
      - ./config.json:/app/config.json:ro
      - ./knowledge_base:/app/knowledge_base:ro
      - ./output:/app/output
    depends_on:
      - ollama
```

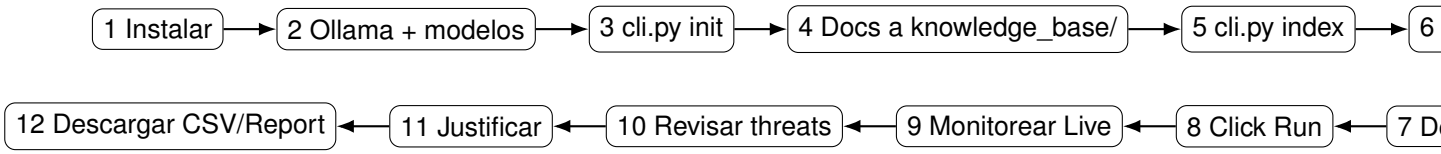
Descargar modelos en Docker

```
docker exec -it agentictm-ollama-1 bash
ollama pull qwen3:8b
ollama pull qwen3:30b-a3b
ollama pull qwen3-vl:8b
ollama pull nomic-embed-text
```

Solo Dockerfile

```
docker build -t agentictm .
docker run -p 8000:8000 \
  -e AGENTICTM_OLLAMA_URL=http://host.docker.internal:11434 \
  -v ./config.json:/app/config.json:ro \
  -v ./output:/app/output \
  agentictm
```

9.6 Flujo completo: de cero a threat model



9.7 Buenas practicas

Input

Mas contexto = mejores resultados. Describir componentes, flujos, trust boundaries y tecnologias.

Knowledge Base

Indexar antes del primer analisis y agregar threat models previos.

Configuracion

`max_debate_rounds: 2-4` para rapido, `6-8` para profundo. `analyst_execution_mode: "hybrid"` como mejor relacion calidad/VRAM.

Performance

Cerrar apps GPU, usar cascade si falta VRAM, primer analisis mas lento por carga de modelos.

Chapter 10

Flujo de Datos

10 – Flujo de Datos

State transitions, que lee y escribe cada agente, y como fluyen los datos por las 6 fases.

10.1 `ThreatModelState` — El Bus de Datos

Todos los agentes comparten un unico `ThreatModelState` (TypedDict) que funciona como bus de datos. LangGraph fusiona updates parciales al state.

```
class ThreatModelState {
    +str system_input
    +str system_name
    +list~str~ threat_categories
    +list~str~ upload_paths
    +str dfd_mermaid
    +list~Component~ components
    +list~DataFlow~ data_flows
    +list~TrustBoundary~ trust_boundaries
    +list~str~ input_images
    +list~dict~ stride_analysis
    +list~dict~ pasta_analysis
    +list~dict~ attack_trees
    +list~dict~ maestro_analysis
    +list~dict~ ai_threat_analysis
    +list~DebateEntry~ debate_history
    +list~dict~ threats_unified
    +list~dict~ threats_validated
    +list~dict~ threats_final
    +str csv_output
    +str report_output
    +str latex_output
    +dict agent_metrics
    +dict config
    +int debate_round
    +int max_debate_rounds
}
```

Campos con `Annotated[list, operator.add]`

Cuando dos nodos escriben al mismo campo, LangGraph concatena listas en lugar de sobrescribir:

```
# Nodo 1: {"stride_analysis": [threat_A, threat_B]}
# Nodo 2: {"pasta_analysis": [threat_C, threat_D]}
# Fan-in: state tiene ambas listas completas
```

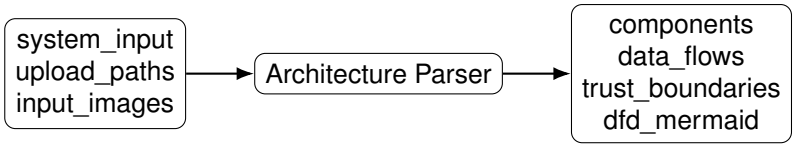
Esto habilita fan-out seguro en modos parallel/hybrid.

10.2 Mapa Lectura/Escritura por Agente

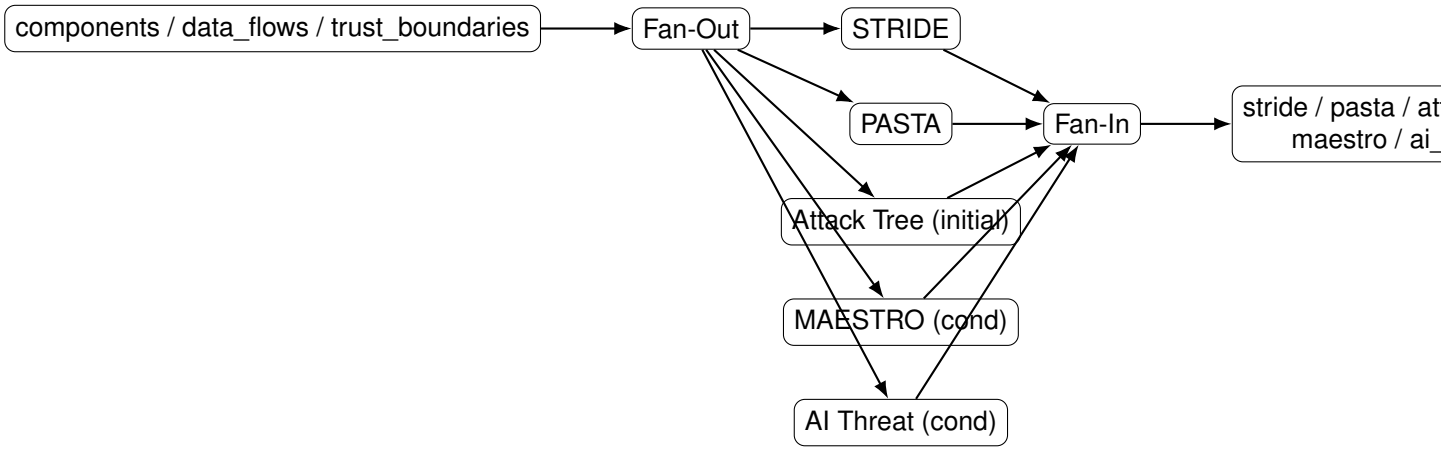
Agente	Lee	Escribe
Architecture Parser	`system_input`, `upload_paths`, `input_images`, `config`	`components`, `data_flows`, `trust_boundaries`, `dfd_mermaid`, `input_images`
STRIDE Analyst	`system_input`, `components`, `data_flows`, `trust_boundaries`, `dfd_mermaid`	`stride_analysis`
PASTA Analyst	`system_input`, `components`, `data_flows`	`pasta_analysis`
Attack Tree Analyst (initial)	`system_input`, `components`, `data_flows`, `stride_analysis`, `pasta_analysis`	`attack_trees`
MAESTRO Analyst	`system_input`, `components`, `data_flows`	`maestro_analysis`
AI Threat Analyst	`system_input`, `components`, `data_flows`	`ai_threat_analysis`
Red Team	`system_input`, `components`, `stride_analysis`, `pasta_analysis`, `attack_trees`, `debate_history`	`debate_history`
Blue Team	`system_input`, `components`, `stride_analysis`, `debate_history`	`debate_history`
Attack Tree Analyst (enriched)	`system_input`, `components`, `attack_trees`, `debate_history`	`attack_trees` (append)
Threat Synthesizer	`stride_analysis`, `pasta_analysis`, `attack_trees`, `maestro_analysis`, `ai_threat_analysis`, `debate_history`, `components`	`threats_unified`
DREAD Validator	`threats_unified` OR `threats_validated`	`threats_validated`
Output Localizer	`threats_validated`, `config`	`threats_final`
Report Generator	`threats_final`, `dfd_mermaid`, `attack_trees`, `debate_history`, `system_input`, `system_name`, `components`	`csv_output`, `report_output`, `latex_output`

10.3 Flujo por Fase

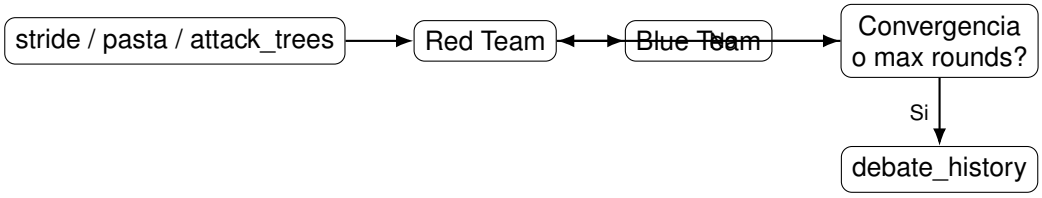
Fase I — Analisis de Arquitectura



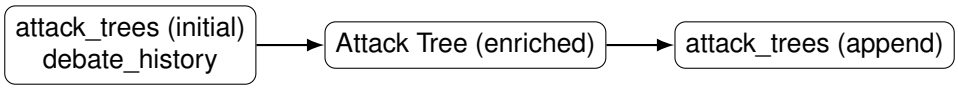
Fase II — Analistas en Paralelo/Hybrid/Cascade



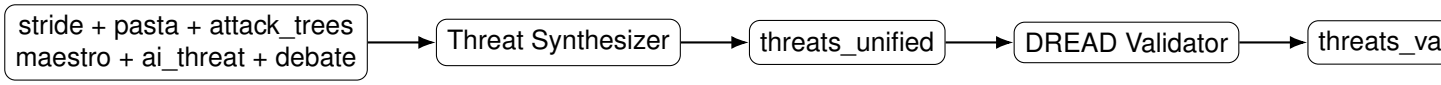
Fase III — Debate Adversarial



Fase II.5 — Attack Trees Enriquecidos

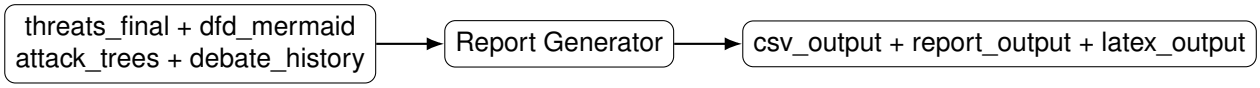


Fase IV — Sintesis y Validacion

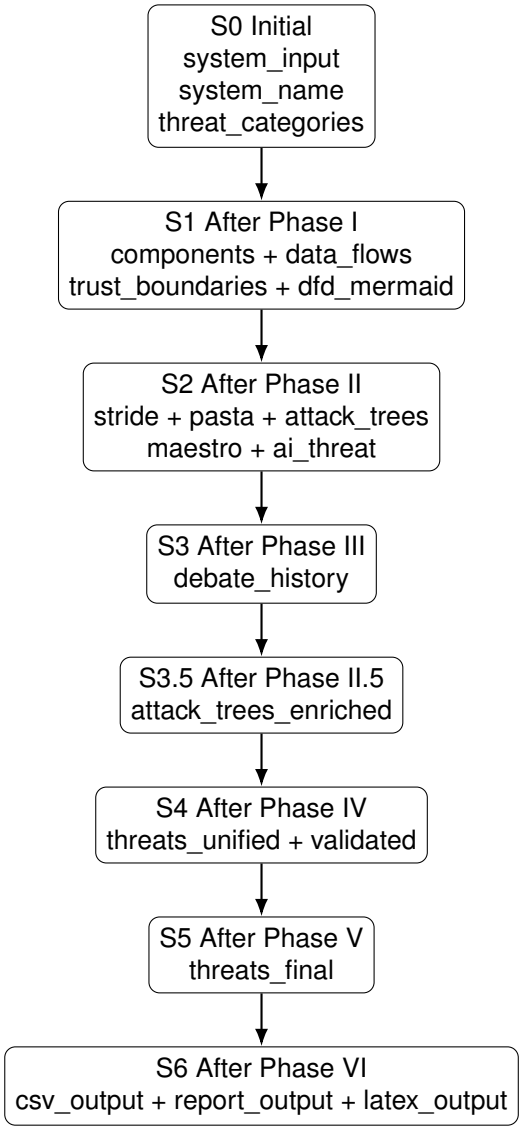


Fase V — Localizacion

Fase VI — Generacion de Reportes



10.4 Diagrama completo: estado a lo largo del pipeline



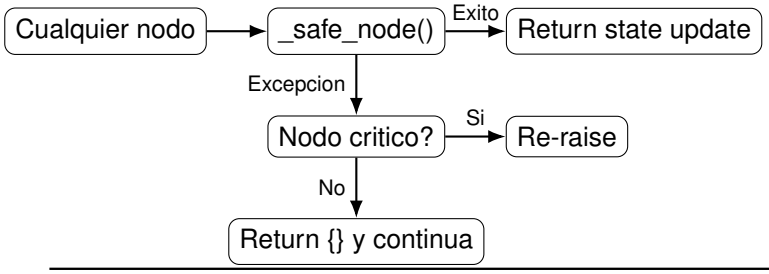
10.5 Volumen de Datos por Fase

Fase	Input (tokens)	Output (tokens)	Campos nuevos
I	1–5K	2–8K	4 campos
II	2–8K x5	2–5K x5	5 campos
III	10–25K x rounds	2–5K x round	1 campo (append)
II.5	5–10K	2–5K	1 campo (append)
IV	15–30K	3–8K -> 3–8K	2 campos
V	3–8K	3–8K	1 campo
VI	5–10K	5–15K	3 campos

Total acumulado

Estado final: 50–100K tokens de datos estructurados.

10.6 Manejo de errores en el flujo



Tipo de nodo	Comportamiento si falla
Critico (`architecture_parser`)	Pipeline aborta.
No-critico (resto)	Degradacion graciosa, el pipeline continua.

Chapter 11

Mejoras y Roadmap

11 – Mejoras y Roadmap

Estado actual, mejoras implementadas, pendientes y vision Oraculus.

11.1 Estado del Proyecto

Aspecto	Estado
Version actual	v0.3.2
Clasificacion	Advanced Prototype / Late Alpha
Pipeline funcional	End-to-end operativo
Frontend	SPA profesional con 7 tabs
API	28+ routes, SSE, persistencia
RAG	Dual (vector + tree), 5 stores
Docker	docker-compose con Ollama
CI/CD	GitHub Actions (lint + test + docker)
Tests	54 tests (pytest)

11.2 Mejoras Implementadas

v0.3.1 — Calidad de output y frontend

ID	Mejora	Impacto
C1	PDF Upload Fix (PyPDFLoader)	PDFs procesados correctamente.
C2	`format="json"` en LLMFactory	Reduce fallos de parseo JSON.
M3	`max_tool_rounds` 3->5	Tools con mas margen.
F1	Tabla de amenazas profesional	Output visual enterprise.
F2	URL routing con pushState	Deep-link support.
F3	Mermaid v11 (DFD + Attack Trees)	Diagramas interactivos.
F4	marked.js + auto-render Mermaid	Reportes profesionales.
F5	LaTeX report generation	Export formal.
F6	SPA routes server-side	Sin 404 en refresh.
F7	Print CSS	Export PDF desde navegador.

v0.3.2 — Pipeline inteligente y debate

ID	Mejora	Impacto
M2	Debate con convergencia	2 rondas si es simple, hasta 10 si complejo.
–	Hybrid execution mode	Balance VRAM/calidad.
–	4-tier LLM (quick/deep/stride/vlm)	Modelo optimo por tarea.
–	STRIDE inference + mitigaciones default	Output consistente.
–	Output Localizer condicional	Traduccion solo si `output_language="es"`.
–	Report Generator en espanol	Headers y categorias en espanol.
–	Evidence sources + evidence notes	Trazabilidad.
–	Stale cache fix frontend	Sin datos residuales.
–	Architecture Parser optimizado	Mas rapido sin perder calidad.

11.3 Mejoras Pendientes

Prioridad critica (C)

ID	Mejora	Esfuerzo	Descripcion
C3	Upload Hardening	Bajo	Limites, MIME whitelist, cleanup.

Prioridad alta (H)

ID	Mejora	Esfuerzo	Descripcion
H1	Diferenciacion real quick vs deep	Bajo	Deep mas capaz en config.
H2	Unificacion idioma prompts	Medio	Unificar a espanol.
H3	Prompt Synthesizer >=15 amenazas	Medio	Minimo 15 amenazas.
H4	Schema unificado de amenazas	Alto	`Threat` Pydantic.
H5	Validacion DREAD calibrada	Medio	Criterios explicitos.
H6	RAG con reranking	Medio	Cross-encoder.
H7	Error handling robusto	Medio	Try/except por nodo.

Prioridad media (M)

ID	Mejora	Descripcion
M1	Usar `max_validation_iterations`	Re-validacion DREAD.
M4	Markdown report con DREAD desglosado	Columnas D/R/E/A/D.
M5	Meta-prompt con TMs previos	Few-shot para Synthesizer.
M6	Endpoint `/api/results/{id}/json`	Descarga state completo.

Prioridad baja (L)

ID	Mejora
L1	Dashboard de metricas.
L2	Comparacion entre analisis.
L3	Integracion Jira.
L4	Multi-idioma en output.
L5	Rate limiting y auth avanzada.
L6	Streaming de resultados parciales.
L7	Tests E2E.
L8	OpenAPI docs.
L9	KB auto-update.
L10	Plugin system.

11.4 Hallazgos de Evaluaciones

Evaluacion profunda (doc 13)

Veredicto: *Very good advanced prototype*. Debate estatico resuelto en v0.3.2, Spanglish resuelto con Output Localizer.

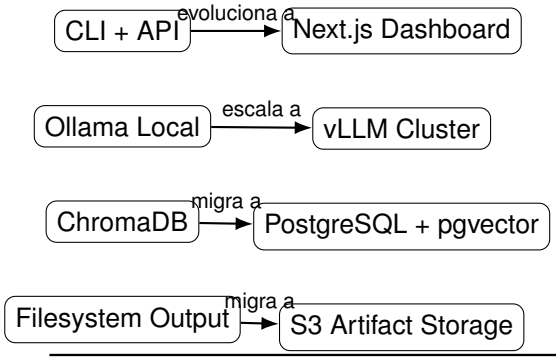
Analisis objetivo (doc 14)

Fan-out brillante, debate estatico y Spanglish degradaban calidad. Implementado en v0.3.2.

Evaluacion independiente (doc 15)

Advanced Prototype / Late Alpha. Fases 1–5 con pendientes H4, M1, L7.

11.5 Vision Oraculus — Enterprise Future



Componente	AgenticTM (Actual)	Oraculus (Futuro)
Frontend	SPA HTML/JS	Next.js + React
LLM	Ollama local	vLLM multi-GPU
Vector DB	ChromaDB	PostgreSQL + pgvector
Storage	Filesystem	S3/MinIO
Auth	API key simple	Keycloak SSO, RBAC
Integrations	–	Jira, SIEM/SOAR, CI/CD
Multi-tenant	No	Si
Compliance	–	SOC2, ISO 27001

11.6 Proximos pasos recomendados

Sprint 1 — Estabilizacion

1. H1: Diferenciar modelos (deep realmente distinto).
2. C3: Upload hardening.
3. H7: Error handling robusto por nodo.

Sprint 2 — Calidad

1. H3: Prompt sintetizador >= 15 amenazas.
2. H2: Unificar idioma de prompts a espanol.
3. M5: Few-shot con TMs previos.

Sprint 3 — Produccion

1. H4: Schema Pydantic unificado.
2. M1: Validation iterations.
3. L7: Tests E2E.

Chapter 12

Troubleshooting

Esta sección documenta errores frecuentes y recomendaciones para el rendimiento operativo.

12.1 Errores Comunes

Ollama y Configuración de Modelos

Error	Causa y Solución
`Cannot connect to Ollama`	Ollama no está corriendo. Iniciar con `ollama serve`.
`Model not found`	Descargar el modelo requerido (`ollama pull qwen3:8b`).
`CUDA out of memory`	VRAM insuficiente. Reducir `num_gpu` o cambiar a `cascade` mode.
`Request timeout`	Aumentar el timeout en `config.json` o usar modelos de menor peso.

Pipeline y API

- **Architecture Parser failed:** Input incompleto o no reconocido. Expandir la descripción del sistema.
- **Debate stuck in loop:** Valor de `max_debate_rounds` muy alto. Reducirlo a 3-4.
- **Port 8000 in use:** Matar procesos huérfanos (`taskkill /F /PID {pid}`).
- **SSE desconectado:** Verificar que el proxy (nginx/Cloudflare) posea `X-Accel-Buffering: no`.

12.2 Estructura de Archivos del Proyecto

Directorio Principal de AgenticTM

```
AgenticTM/
|-- cli.py # CLI Typer (analyze, index, init)
|-- run.py # Server launcher (uvicorn)
|-- config.json # Configuracion del proyecto
|-- agentictm/
| |-- config.py # Pydantic config models
| |-- core.py # AgenticTM orchestrator class
| |-- agents/ # 12 Nodos Analistas y Sintetizadores
| |-- graph/ # Construccion de LangGraph
| |-- llm/ # LLMFactory (7 properties)
| |-- api/ # FastAPI (28+ routes)
| `-- rag/ # RAGStoreManager y tools
|-- knowledge_base/ # PDF, CSV, y archivos base
|-- data/ # Bases de datos, ChromaDB, indices
```

```
`-- output/ # Salidas de Modelos y Reportes
```

Chapter 13

Evaluación Profunda de AgenticTM y Plan de Acción

Nota del 2026-02-24: Esta evaluación profunda audita el grado de "production-readiness" del producto, emitiendo un veredicto técnico y estratégico.

13.1 Veredicto Resumido

El sistema se posiciona como un **prototipo avanzado muy bueno** de threat modeling multi-agente (con arquitectura sólida, alta cobertura y salida útil), pero requiere ajustes en 4 ejes críticos antes de ser categorizado como *production-grade*: Ingesta documental robusta, Gobernanza de outputs (Validación estructurada), Observabilidad, y Benchmarking de calidad.

13.2 Deudas Técnicas Prioritarias (P0 y P1)

- **Parsing de PDF en Uploads:** Los archivos PDF deben ser parseados con herramientas robustas (como `PyPDFLoader`), en lugar de leerse binariamente como texto.
- **Contratos JSON Estrictos:** Los esquemas de salida varían entre agentes. Es imperativo utilizar herramientas nativas de los LLMs como `with_structured_output` de LangChain.
- **Diferenciación de hardware (Tiers):** Actualmente *quick* y *deep thinker* operan con el mismo modelo (`qwen3:8b`). Esto no explota las bondades de tareas analíticas pesadas que requieren el `qwen3:30b-a3b`.
- **Prompt Engineering:** Limpiar la hibridación Español-Inglés para evitar *cognitive overload* y contaminación en el idioma de salida.

Chapter 14

Análisis Objetivo y Roadmap de Evolución

Reporte de Análisis, Fecha: 2026-02-24

14.1 Análisis de Componentes

- **Flujo LangGraph:** El patrón "Fan-out / Fan-in" es brillante. No obstante, el debate del Red y Blue Team adolecía de inflexibilidad (estático por rondas), lo que en versiones recientes comenzó a corregirse con un enfoque dinámico y señales de convergencia.
- **Código y Robustez:** Se abusa de heurísticas frágiles basadas en Expresiones Regulares para sanear JSONs rotos desde los agentes.
- **Habilidades Agénticas:** Fuerte base en el uso de herramientas (RAG), pero falta la capa de *Autocorrección* (Self-Reflection) tras fallar una instrucción.
- **Modelos LLM:** (Reafirmando evaluaciones anteriores) Es la mayor barrera técnica actual: el cuello de botella cognitivo del `qwen3:8b` limitando la Síntesis.

14.2 Roadmap Estratégico

1. **Fase 1 (Corto Plazo):** Arreglar el parsing de PDF de los usuarios, traducir los System Prompts 100% a inglés (demandando el output final en español) y habilitar una diferenciación real de modelos locales en el `config.json`.
2. **Fase 2 (Mediano Plazo):** Incorporar Cross-Encoders/Rerankers para el RAG y forzar Pydantic Schemas a nivel de la API del LLM.
3. **Fase 3 (Largo Plazo):** Implementar nodos "Jueces" para el debate y obligar a todos los agentes a rastrear con alta precisión el campo `evidence_source` apoyando sus afirmaciones en los PDFs del usuario.

Chapter 15

Evaluación Independiente de AgenticTM

El siguiente análisis fue generado mediante una auditoría exhaustiva y ciega del código fuente (Claude Opus 4.6, 2025-07-15), enfocándose en vulnerabilidades operativas, madurez del RAG y robustez agéntica.

15.1 Análisis de Seguridad y Hardening

Sorprendentemente, para ser una herramienta de ciberseguridad, su infraestructura posee faltas mitigables:

- **Uploads sin límites:** No existen mecanismos de validación real de tamaños máximos, Mime-types, Rate-Limiting ni prevención de path-traversal en la versión analizada.
- **Limpieza de archivos:** Los archivos en la carpeta de adjuntos temporales no están implementando políticas de reciclaje o depuración segura.
- **Ataques de Prompt Injection:** Al leer archivos indiscriminadamente, un documento provisto por el usuario puede incluir instrucciones que fuercen a los agentes a desviar su atención ("Ignore all previous instructions...").

15.2 Observabilidad y UX

A nivel log y métricas, cada agente documenta su trabajo a fondo (registrando invocaciones y latencias). Sin embargo, el frontend y la API carecen de endpoints globales que revelen, por ejemplo, el porcentaje general de éxito/fallo de la decodificación JSON en cada análisis o los costos simulados por uso de tokens.

Chapter 16

Lista Priorizada de Mejoras

Consolidación final de la hoja de ruta de la versión 0.3.1 hacia un entorno estable y Enterprise.

16.1 Leyenda de Prioridad

Nivel	Significado	Criterio de Evaluación
C	Crítico	Bug funcional, output corrupto, bloquea el uso.
H	Alto	Impacta drásticamente la calidad y fiabilidad de amenazas.
M	Medio	Performance, mejoras notables en la UX y consistencia.
L	Bajo	Nice-to-have, métricas y pulido de documentación.

16.2 Checklist de Tareas

Nivel Crítico (C) - Requieren acción inmediata

- **[Implementado] C1: PDF Upload Parsing Bug.** El uso de `PyPDFLoader` se corrigió en el backend.
- **[Implementado] C2: Structured JSON Output.** Activación oficial de `format="json"` y delegación explícita a LangChain.
- **[Pendiente] C3: Upload Hardening.** Limitar los megabytes por sesión, forzar whitelists de MIME types y depuración automática.

Nivel Alto (H) - Fiabilidad

- **[Pendiente] H1: Diferenciación Real.** Habilitar de forma predeterminada modelos 32b/14b para el deep/stride thinker.
- **[Pendiente] H2: Unificación de Prompts.** Ajustar los System Prompts a un idioma canónico con outputs forzados al español.
- **[Pendiente] H3 & H4: Schemas y Sintetizador.** Obligar Pydantic unificado de ‘Threat‘ en todas las fases y solicitar obligatoriamente más de 15 amenazas al Synthesizer para emular un reporte humano.

Niveles Medio y Bajo (M y L) - Expansión

- **[Implementado] M2 y M3:** El Debate tiene señales nativas de convergencia; los analistas ejecutan más rondas (5) por las Tools del RAG.

- **[Pendiente] L1 al L10:** Desarrollo de APIs documentadas en OpenAPI, Tests End-to-End, exportación e integración a JIRA, y actualizaciones automáticas con bases CVE / ATT&CK.